

DeepVision Studio 开发文档

本文档用于课程项目答辩与后续维护，不按开发时间顺序记录，而是按项目架构、模块设计、实现难点、部署方案与小组分工组织。

1. 文档定位

DeepVision Studio 是一个面向深度学习教学的可视化实验平台，包含网络结构编辑、真实前向计算、训练实验、可视化解释、AI 辅助问答、在线协作与部署。

本文档采用以下写法：

- 以架构与功能模块为主线，说明系统为什么这样设计。
- 每个成员的贡献都区分“人工主导”和“AI 辅助”，AI 不作为独立作者，而是归属到具体成员的研发过程。
- 对工程中可能遇到的关键问题给出合理复盘，包括问题背景、解决策略与代码落点。
- 按成员实际负责范围记录 A/B/C/D/E/F 模式、训练协作、解释器与部署模块，具体描述以仓库当前代码为准。

2. 项目概览

2.1 项目名称

DeepVision Studio：深度学习算法可视化仿真平台。

2.2 核心能力

能力	说明	主要代码位置
首页与模式入口	统一进入 A/B/C/D/E/F、教学文档与 AI 博物馆	frontend/src/app/shell/home
A 模式：前向传播实验室	编辑 CNN 风格网络、选择样本、执行真实前向传播、观察每层输出	frontend/src/app/modes/mode-a
B 模式：模型训练工作台	数据集管理、结构编辑、真实 PyTorch 训练、指标与反向传播观察	frontend/src/app/modes/mode-b , backend/spring/src/main/java/com/deepvision/studio/training , backend/python-training
B 模式实验对比	按数据集检索历史 checkpoint，对比结构、超参数、训练曲线与结果状态	frontend/src/app/modes/mode-b/experiment-compare
B 模式单样本推理	从已完成 checkpoint 选择样本，执行真实推理并展示逐层激活	frontend/src/app/modes/mode-b/single-inference

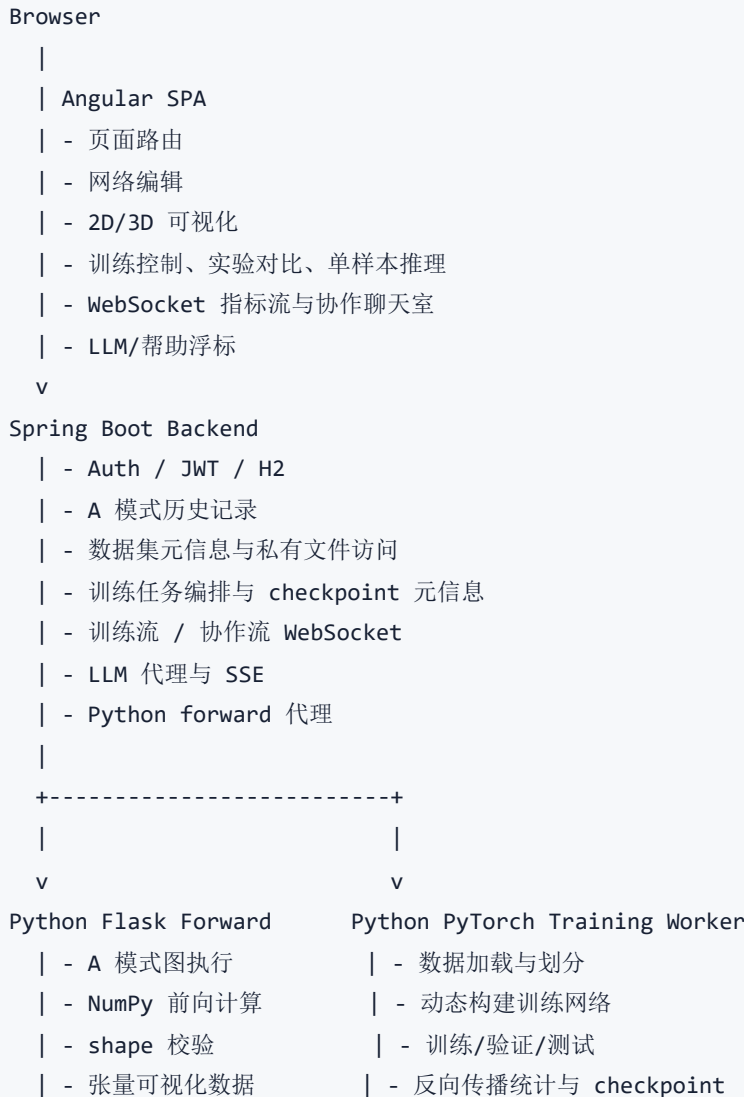
能力	说明	主要代码位置
B 模式训练协作	独立聊天室、在线成员、训练状态旁观、日志同步与智能助手	frontend/src/app/modes/mode-b/training-collaboration , backend/spring/src/main/java/com/deepvision/studio/training/TrainingCollaborationHandler.java
C 模式：CNN 卷积解释器	浏览器端加载 CNN Explainer 资源与 TensorFlow.js 模型，展示样例、网络拓扑、中间特征图和卷积层细节	frontend/src/app/modes/mode-c , frontend/src/app/modes/mode-c/explainer , frontend/public/mode-c/cnn-explainer
C 模式卷积过程解释	支持输入 patch、kernel 权重、逐元素乘积、加权求和、bias、输出格回流，以及 ReLU、Pooling、Softmax 解释	frontend/src/app/modes/mode-c/explainer/components/detail-panels , frontend/src/app/modes/mode-c/explainer/components/overview
C 模式 Grad-CAM 与解释报告	基于真实 tfjs CNN 模型生成 Grad-CAM 热力图、原图叠加、主导通道摘要和可演示解释报告	frontend/src/app/modes/mode-c/explainer/services/mode-c-inference.service.ts , frontend/src/app/modes/mode-c/explainer/components/article
D 模式：Transformer 解释器	浏览器端加载 GPT-2 ONNX/tokenizer/wasm 资源，执行下一词预测并展示 Top-K 概率	frontend/src/app/modes/mode-d , frontend/public/mode-d-assets , scripts/sync-transformer-explainer.ps1
D 模式注意力矩阵	支持层/头选择、真实 attention matrix 可视化、hover/click 聚焦和当前注意力单元解释	frontend/src/app/modes/mode-d/explainer/components/attention-matrix , frontend/src/app/modes/mode-d/explainer/services/mode-d-state.service.ts
D 模式 QKV 教学可视化	将 Query、Key、Score、Value、Output 串成交互式教学链，展示向量维度联动、softmax 权重和 Value 汇流贡献	frontend/src/app/modes/mode-d/explainer/components/qkv-panel
D 模式解释报告	汇总当前输入、Top-K 预测、注意力焦点、QKV 过程和自动解释文本	frontend/src/app/modes/mode-d/explainer/components/report-panel
E 模式：反向传播可视化	纯 TypeScript 实现 MLP 前向、损失、反向传播、参数更新、子步骤动画和决策边界	frontend/src/app/modes/mode-e , frontend/src/app/modes/mode-e/explainer/engine/mode-e-backprop-engine.ts
E 模式优化器与曲线对比	支持 SGD、Momentum、Adam、学习率调节、激活函数切换、损失曲线保存/替换/删除和决策边界对比	frontend/src/app/modes/mode-e/explainer/services/mode-e-state.service.ts , frontend/src/app/modes/mode-e/explainer/components/floating-charts , frontend/src/app/modes/mode-e/explainer/components/control-panel

能力	说明	主要代码位置
F 模式： RNN 与 BPTT 解 释器	纯 TypeScript 实现 RNN 前向、时间反向传播、序列分类预设、隐状态时间展开图和梯度范数展示	frontend/src/app/modes/mode-f , frontend/src/app/modes/mode-f/explainer/engine/mode-f-rnn-engine.ts
AI 博物馆	第一人称漫游 AI 发展史长廊，支持多人联机同馆参观	frontend/src/app/modes/ai-museum , backend/spring/src/main/java/com/deepvision/studio/museum
3D 网络 显示	将网络层、shape、特征图快照映射为 Three.js 3D 场景	frontend/src/app/shared/network-3d
登录注册	用户注册、登录、JWT 会话恢复	frontend/src/app/core/auth , backend/spring/src/main/java/com/deepvision/studio/auth
A 模式历史 记录	保存网络快照、预览图、参数统计，并支持回溯	frontend/src/app/shared/forward , backend/spring/src/main/java/com/deepvision/studio/forward
LLM 浮标	页面上下文问答、图像上下文、流式输出	frontend/src/app/shared/llm , backend/spring/src/main/java/com/deepvision/studio/llm
教学帮助 浮标	术语高亮、术语检索、教学文档跳转	frontend/src/app/shared/teaching , frontend/src/app/shell/teaching
后端网关	Spring Boot 统一承接认证、持久化、LLM、forward 代理、训练任务与 WebSocket	backend/spring
Swagger /OpenA PI 文档	Spring REST 接口分组、请求/响应模型和 JWT 说明	backend/spring/src/main/java/com/deepvision/studio/common/OpenApiConfig.java
Python forward 服务	使用 NumPy 执行真实前向传播计算	backend/python-forward
Python training worker	使用 PyTorch 加载数据、构建网络、训练、测试、保存 checkpoint，并输出结构化事件	backend/python-training/training_worker.py
Docker 部署	前端 Nginx、Spring 后端、Python forward 三容器编排；训练 worker 随 Spring 镜像安装并按任务启动	docker-compose.yml , frontend/Dockerfile , backend/spring/Dockerfile , backend/python-forward/Dockerfile

3. 总体架构设计

3.1 架构分层

项目采用“前端可视化 + Spring 业务后端 + Python 计算服务”的分层方式:



这样拆分的原因:

- Angular 适合承载复杂交互、局部状态和可视化组件。
- Spring Boot 负责认证、安全、接口、数据库、异常处理、部署配置。
- Python/NumPy 负责 A 模式前向计算, Python/PyTorch 负责 B 模式训练和 checkpoint 推理, 避免在 Java 或 TypeScript 中重复实现数值计算与自动求导。
- Spring 作为唯一业务入口代理 Python 服务, 前端不直接访问 Python, 部署时容器网络和安全边界更清晰。
- B 模式训练 worker 不是常驻 HTTP 服务。Spring 为每个训练任务生成 `request.json` 和 `control.json`, 再通过 `ProcessBuilder` 启动 Python 子进程, 并把标准输出中的 JSON 事件转发到 WebSocket。

3.2 前端架构

前端按“通用基础设施、共享能力、模式页面、外壳页面”分层:

目录	职责
core	全局 API、认证、登录注册页
shared	多模式复用能力，如网络图、3D 展示、LLM 浮标、教学浮标、forward 客户端
modes	A/B/C/D/E 各模式页面与其私有逻辑
shell	首页、教学文档等非模式页面
public	大体积静态资源、样本图片、第三方解释器资源

路由入口在 `frontend/src/app/app.routes.ts`，主要路由包括：

路径	页面
<code>/</code>	首页
<code>/login</code> 、 <code>/register</code>	登录注册
<code>/mode-a</code>	A 模式前向传播实验室
<code>/mode-b</code>	训练模式
<code>/training/experiments</code>	B 模式实验对比
<code>/training/inference</code>	B 模式单样本推理
<code>/training/collaboration</code>	B 模式训练协作聊天室
<code>/mode-c</code>	CNN 解释器
<code>/mode-d</code>	Transformer 下一词预测与注意力解释器
<code>/mode-e</code>	反向传播可视化
<code>/mode-f</code>	RNN 循环神经网络解释器
<code>/ai-museum</code>	AI 博物馆
<code>/network-3d</code>	3D 网络显示窗口
<code>/teaching</code>	教学文档

3.3 后端架构

Spring Boot 后端按业务域划分包：

包	职责
auth	用户、登录注册、JWT、UserDetails
common	安全配置、CORS、静态资源映射、健康检查、统一异常
forward	A 模式 forward 代理、历史记录、预览图保存

包	职责
training	训练数据集、任务编排、检查点、单样本推理、训练指标 WebSocket 与协作聊天室
llm	大模型聊天代理与 SSE 流式输出
museum	AI 博物馆在线状态 WebSocket
common/OpenApiConfig	Spring REST API 的 OpenAPI 元信息和 JWT Bearer 鉴权说明

数据库当前使用 H2 文件数据库，配置在 backend/spring/src/main/resources/application.yml。实体设计基于 JPA，后续可替换为 MySQL/PostgreSQL。

3.4 Mode B 的接入方式

Mode B 是项目中跨层最多的业务模块，覆盖 Angular 页面、Spring 任务编排、H2 元信息、文件系统数据、PyTorch worker 和两类 WebSocket。其主链路如下：

Angular Mode B

-> GET/POST /api/training/datasets/** 管理训练数据

-> POST /api/training/start 提交数据划分、网络层和超参数

-> Spring 创建 training-jobs/<jobId>/request.json、control.json

-> Spring 启动 python-training/training_worker.py

-> Python 加载 datasets/**，构建 PyTorch 模型并训练

-> stdout 输出 metric/backprop/test_result/control/error JSON

-> Spring 缓存并通过 /api/training/stream WebSocket 推送

-> Angular 更新训练状态、曲线、日志和反向传播面板

-> Python 保存 checkpoint.pt

-> Spring 将 checkpoint 路径及实验元信息写入 H2

Mode B 的存储采用“数据库保存可检索元信息，文件系统保存大文件”的方式：

存储位置	内容	原因
H2 training_datasets	数据集名称、类型、所有者、样本数、类别数、预览与告警 JSON	支持按用户筛选和快速展示，不把大体积样本写入数据库
datasets/builtin/**	内置图片和 CSV 文件	供预览和 Python worker 直接读取
datasets/upload/<datasetId>/**	用户上传的 CSV、图片、标签列和类别数辅助文件	保持原始训练文件结构，便于 PyTorch Dataset 加载
H2 training_checkpoints	用户、数据集、网络层、超参数、划分、曲线、指标、权重文件路径	支持实验检索、对比和权限隔离
training-jobs/<jobId>/checkpoint.pt	PyTorch state_dict、网络层、配置、类别数和模型签名	二进制权重体积较大，文件存储更适合直接由 PyTorch 加载

训练指标流和协作流分开设计：

- `/api/training/stream` 只允许持有 JWT 且拥有该训练任务的用户订阅。
- `/api/training/collaboration` 负责房间、聊天、在线成员和 @智能助手。
- `/api/training/collaboration/stream` 只向已经加入房间的参与者转发训练事件，使旁观者不获得暂停、停止等任务控制权限。

3.5 Mode C / Mode D 的接入方式

成员 C 负责的 Mode C 与 Mode D 没有沿用模式 A 的 “Angular -> Spring -> Python” 推理链，而是采用了**浏览器端原生推理 + 平台共享能力接入**的方式：

模式	推理位置	主要运行时	静态资源来源	与 Spring 的关系
Mode C	浏览器端	TensorFlow.js	frontend/public/mode-c/cnn-explainer/**	不通过 Spring 执行推理；仅复用登录状态、LLM 代理与教学入口
Mode D	浏览器端	onnxruntime-web + @xenova/transformers	frontend/public/mode-d-assets/**	不通过 Spring 执行推理；仅复用登录状态、LLM 代理与教学入口

这两条模式的接入思路与模式 A/B 的区别在于：

- 推理数据不经过 Spring 或 Python 服务中转，而是在 Angular 页面内部直接加载模型与样例资源。
- 页面级状态由各自的 StateService 管理，负责串联样例切换、推理结果、解释视图与 AI 上下文。
- 与后端的耦合点集中在平台共用能力：认证接口 `api/auth/**`、LLM 代理接口 `api/llm/**`，而不是单独的模式业务接口。
- Mode C 的外部依赖主要是迁移后的 CNN Explainer 静态资源；Mode D 的外部依赖主要是 GPT-2 ONNX 分片、transformers 运行时与 onnxruntime wasm 文件，因此额外提供了资源同步脚本。

从运行链路上看，二者分别形成了如下结构：

Mode C

Angular Component

- > ModeCStateService
- > ModeCAssetsService 读取模型配置、样例和静态资源路径
- > ModeCInferenceService 加载 `tf.js` 与 `model.json`
- > 浏览器内执行 CNN 前向推理 / Grad-CAM / 通道级解释
- > Overview + Detail Panel 渲染结果

Mode D

Angular Component

- > ModeDStateService
- > ModeDAssetsService 提供样例、层/头选项
- > ModeDInferenceService 加载 `tokenizer` / `ONNX session` / `attention tensor`
- > 浏览器内执行 `next-token` 推理
- > Top-K / Attention Matrix / QKV / Report 渲染结果

因此，在项目总体架构上，Mode C 和 Mode D 补充了平台中的第二类模式形态：一类是依赖 Spring/Python 服务的服务端推理模式（如模式 A、模式 B）；另一类是依赖静态模型资源与浏览器端运行时的前端原生解释模式（如模式 C、模式 D）。

4. 小组分工与 AI 使用标注

4.1 标注规则

标记	含义
人工主导	需求拆解、架构取舍、关键逻辑、调试与集成由成员完成
AI 辅助	使用 AI 辅助生成样式草稿、接口样板、文案、局部算法参考或排错建议
人工复核	对 AI 产出进行修改、测试、合并、删减和工程化落地

4.2 成员总览

成员	代号	主要职责	人工/AI 比例说明
王龄锋	成员 A	项目初始化、整体网站设计、A 模式、登录注册、H2 数据库、LLM 浮标、帮助浮标、Spring/forward 服务、Docker 部署与云端部署	人工主导架构与核心逻辑，AI 辅助 UI 细化、接口样板和文档整理；云端部署由人工完成，无 AI 参与
李子涵	成员 B	B 模式训练工作台、数据集与数据库、PyTorch 训练链、checkpoint、实验对比、单样本推理、训练协作聊天室及相关页面样式	人工确定模块边界、数据结构、接口契约和调试验收；AI 辅助生成部分组件/DTO/WebSocket 样板、可视化草稿和报错排查建议，最终由人工整合修改
肖羽平	成员 C	模块 C（CNN 卷积过程解释）与模块 D（Transformer 注意力/QKV 解释）、第三方解释器迁移、浏览器端推理接入、平台共享能力复用	人工确定模块边界、状态结构、推理链路和交互形式；AI 辅助生成局部组件样板、部分可视化实现参考、报错排查与文档整理，最终代码由人工集成调试与验收
赵红林	成员 D	模式 E（MLP 反向传播可视化引擎、子步骤动画、决策边界、优化器对比）、模式 F（RNN + BPTT 引擎、时间展开图、序列分类数据集）、12 条教学术语、E/F 模式的 AI 助手接入	人工主导引擎公式推导与实现、状态机设计、架构选型与调试验收；AI 辅助组件样板生成、CSS 样式微调、批量重命名与构建排错

5. 成员 A：王龄锋开发内容

5.1 工作范围

成员 A 主要负责项目基础框架和 A 模式相关模块：

模块	具体内容	贡献方式
项目初始化	建立前后端目录结构，确定 Angular + Spring Boot + Python 服务拆分	人工主导
模式设计	规划 A/B/C/D/E 多模式平台结构，A 模式优先完整实现	人工主导，AI 辅助整理交互草图

模块	具体内容	贡献方式
A 模式页面	网络模板、层编辑、样本选择、前向计算、层检查器、卷积核对比、历史记录	人工主导核心交互与数据流，AI 辅助局部样式和提示文案
3D 网络层显示	Three.js 场景、层几何体、连接线、传播粒子、交互选中、层详情面板	人工主导方案和集成，AI 辅助部分 Three.js API 写法
AI 博物馆	第一人称 AI 发展史展厅、展品路线、展墙内容、多人联机在线状态	人工主导产品设计和 WebSocket 方案，AI 辅助展品文案与 Three.js 局部实现
Spring 后端	认证、JWT、安全配置、forward 代理、历史记录、上传图保存、LLM 代理	人工主导接口设计与落地
Swagger 接口文档	Spring REST Controller 注解、DTO Schema、Swagger UI 放行配置	人工主导接口分组和描述，AI 辅助注解样板
H2 数据库	用户表、A 模式历史记录表、JPA 实体与索引	人工主导
登录注册	前端登录注册页、会话恢复、后端密码加密和 JWT	人工主导，AI 辅助表单样式
LLM 浮标	浮动聊天窗口、页面上下文、图像上下文、流式响应	人工主导功能设计，AI 辅助 Markdown 渲染与 prompt 文案
帮助文档浮标	术语高亮、教学文档入口、浮标交互	人工主导
Docker 部署	三服务容器、环境变量、数据卷、Nginx 代理	人工主导，AI 辅助命令说明
云端部署	将 Docker 化项目部署到 Ubuntu 22.04 云服务器，公网通过 <code>http://1.117.223.242/</code> 访问，Nginx 对外暴露 80 端口并代理 <code>/api/**</code> 到 Spring；Spring 和 Python forward 仅在 Docker 网络内通信	人工完成，无 AI 参与

5.2 A 模式设计思路

A 模式定位为“前向传播实验室”，将输入图像、每层输出 shape、每层张量可视化、Top-K 结果和公式说明放在同一个操作界面中。

前端主页面位于 `frontend/src/app/modes/mode-a/mode-a-page.component.ts` 与 `.html`。页面被拆成三栏：

区域	功能
左侧面板	输入样本、图像预处理、网络模板选择
中间画布	网络结构编辑、层拖拽/新增/删除、参数编辑、3D 显示入口
右侧检查器	当前层 shape、公式、特征图、通道预览、统计值、最终输出

核心数据流如下：

```
用户选择样本/调整网络
-> 前端生成 layers + connections + inputTensor
-> POST /api/forward
-> Spring 代理到 Python /api/forward
-> Python NumPy 执行图计算
-> 返回 layerResults / finalTensor / stats / validationIssues
-> 前端渲染每层输出、公式、图像、Top-K、shape path
```

5.3 A 模式网络结构与前向计算

A 模式支持的层类型包括：

层类型	作用
input	输入图像张量，记录预处理方式
conv2d	卷积，支持 kernel、stride、padding、dilation、activation、多输入/输出通道
pool2d	最大池化或平均池化
flatten	将图像张量转为向量
dense	全连接层
activation	独立激活层
dropout	推理/训练状态下的 dropout 演示
output	输出层与 Top-K 类别展示

真实前向计算放在 backend/python-forward/forward_engine.py ，主要流程为：

- 1. 构建执行图：根据 layers 和 connections 建立入边、出边。
- 2. 拓扑排序：检查循环依赖或非法连接。
- 3. 参数校验：对 kernel、stride、padding、units、dropout rate 等字段进行约束。
- 4. 逐层执行：调用对应 operator，如 run_conv2d_operator、run_pool2d_operator、run_dense_operator。
- 5. 生成可视化数据：输出 shape、张量统计、通道预览、Top-K。

其中卷积计算使用 NumPy 的 sliding_window_view 和 tensordot ，减少 Python 层循环。Dense 层在没有手动权重时使用基于层 ID 的确定性权重生成，保证同一结构重复运行结果稳定。

5.4 Spring forward 代理设计

前端只调用 Spring，不直接调用 Python 服务。对应接口在 `backend/spring/src/main/java/com/deepvision/studio/forward/ForwardProxyController.java`：

方法	路径	说明	鉴权
GET	<code>/api/forward/health</code>	检查 Python forward 服务状态	公开
POST	<code>/api/forward</code>	转发前向传播请求到 Python <code>/api/forward</code>	公开

代理层做了三件事：

- 将 Python 服务地址抽成配置项 `deepvision.forward.base-url`，本地和 Docker 环境使用不同地址。
- 设置连接超时与读取超时，避免前端请求无限挂起。
- 捕获 Python 服务不可用和 HTTP 异常，返回更适合前端显示的错误信息。

Python 服务自身接口在 `backend/python-forward/app.py`：

方法	路径	输入	输出
GET	<code>/api/health</code>	无	<code>{ ok, service }</code>
POST	<code>/api/forward</code>	<code>layers</code> 、 <code>connections</code> 、 <code>inputTensor</code>	<code>ForwardPassResult</code> 风格的 JSON

5.5 A 模式历史记录与 H2 数据库

为了让 A 模式不只是一次性演示，成员 A 设计了历史记录功能。用户登录后可以保存当前网络结构、输入样本、前向传播结果和预览图，并在以后回溯。

相关后端代码：

- `ForwardRecordController.java`
- `ForwardRecord.java`
- `ForwardRecordRepository.java`
- `LocalImageStorage.java`
- `ForwardRecordDtos.java`

数据库实体设计：

表/实体	字段	说明
<code>app_users</code> / <code>AppUser</code>	<code>id</code> 、 <code>username</code> 、 <code>passwordHash</code> 、 <code>displayName</code> 、 <code>createdAt</code>	用户账户
<code>forward_records</code> / <code>ForwardRecord</code>	<code>id</code> 、 <code>user_id</code> 、 <code>name</code> 、 <code>templateId</code> 、 <code>datasetName</code> 、 <code>layerCount</code> 、 <code>parameterCount</code> 、 <code>imagePath</code> 、 <code>snapshotJson</code> 、 <code>createdAt</code>	A 模式保存记录

`forward_records` 通过 `user_id` 关联用户，并建立 `user_id`, `created_at` 索引，用于按用户倒序读取历史记录。

接口设计：

方法	路径	说明	鉴权
GET	/api/a/forward-records	获取当前用户 A 模式历史记录列表	JWT
POST	/api/a/forward-records	保存当前 A 模式快照	JWT
GET	/api/a/forward-records/{id}	读取某条记录详情并回溯	JWT
DELETE	/api/a/forward-records/{id}	删除某条记录	JWT

保存时前端会提交 `snapshot` JSON 和 `previewImageDataUrl`。后端将图片从 Data URL 解码为本地文件，路径通过 `/uploads/**` 静态映射访问；完整网络快照则保存为 `snapshotJson`，保持模式结构的灵活性。

5.6 登录注册与安全设计

登录注册由 `auth` 包实现：

接口	说明
POST /api/auth/register	注册用户，密码使用 BCrypt 加密
POST /api/auth/login	登录并签发 JWT
GET /api/auth/me	读取当前用户信息

安全配置位于 `SecurityConfig.java`：

- 使用 `BCryptPasswordEncoder` 存储密码哈希。
- 使用 JWT 进行无状态认证，避免服务端 session。
- 允许前端本地开发端口 `4200/4201/4202` 跨域。
- H2 Console、健康检查、forward、LLM 等接口按开发环境配置放开；A 模式历史记录仍需要用户身份。

前端认证逻辑位于 `frontend/src/app/core/auth`。A 模式页面也内置了保存记录时的登录弹窗，避免用户为了保存实验记录必须先离开当前页面。

5.7 Swagger/OpenAPI 接口文档

Spring 后端引入 `springdoc-openapi-starter-webmvc-ui`，只对 Spring REST 接口生成 Swagger/OpenAPI 文档，Python forward 服务不单独生成 Swagger。

相关代码：

- `backend/spring/pom.xml`：加入 `springdoc-openapi-starter-webmvc-ui`。
- `common/OpenApiConfig.java`：配置 API 标题、版本、服务器地址和 JWT Bearer 鉴权方案。
- `common/SecurityConfig.java`：放行 `/v3/api-docs/**`、`/swagger-ui/**`、`/swagger-ui.html`。
- 各 REST Controller：使用 `@Tag`、`@Operation`、`@ApiResponse` 描述接口分组、用途和返回状态。
- 主要 DTO：使用 `@Schema` 描述请求体和响应模型。

Swagger 分组覆盖：

分组	代码位置	内容
Health	<code>HealthController</code>	Spring 健康检查

分组	代码位置	内容
Auth	AuthController	注册、登录、当前用户
Mode A Forward	ForwardProxyController	Python forward 健康检查和前向计算代理
Mode A Records	ForwardRecordController	A 模式历史记录的列表、保存、详情、删除
LLM	LlmController	普通聊天和 SSE 流式聊天
Training	TrainingController	数据集、训练任务、checkpoint、实验控制和协作房间查询

本地访问地址：

```
http://127.0.0.1:8080/swagger-ui/index.html
http://127.0.0.1:8080/v3/api-docs
```

Docker 或 Nginx 代理后，可通过前端同源地址访问：

```
http://localhost:4200/swagger-ui/index.html
http://localhost:4200/v3/api-docs
```

其中需要用户身份的接口在 OpenAPI 中使用 `bearerAuth` 标记，调用时在 Swagger UI 的 Authorize 中填入登录接口返回的 JWT。

5.8 3D 网络层显示

3D 网络显示位于 `frontend/src/app/shared/network-3d`，入口路由为 `/network-3d`。A 模式点击“3D化显示”时，会把当前网络快照写入 `sessionStorage`，再打开独立窗口展示。

设计重点：

- 使用 Three.js 渲染网络层，避免把 3D 结构塞进普通 DOM 导致性能和透视关系难以控制。
- 根据层类型选择不同几何表达：卷积/池化为特征图堆叠，Flatten 为条带，Dense/Output 为单元网格。
- 把前向传播结果中的特征图预览贴到 3D 平面上，使 3D 场景不只是结构图，而能显示真实计算结果。
- 增加 OrbitControls、单击选中、双击聚焦、传播粒子和右侧详情面板。

工程难点与解决：

难点	解决方式
不同层 shape 差异很大，直接按真实尺寸渲染会过大或过小	在 <code>network-3d-layout.ts</code> 中将 shape 映射为受限宽高深，保留比例但限制视野
特征图通道多，全部渲染会卡顿	最多展示部分通道，并用 <code>+N ch</code> 标记剩余通道
3D 场景与 A 模式页面状态解耦	使用 <code>NETWORK_3D_SESSION_KEY</code> 传递快照，独立窗口只读快照
Three.js 资源泄漏	组件销毁时 <code>dispose</code> <code>renderer</code> 、 <code>controls</code> 、 <code>geometry</code> 、 <code>material</code>

5.9 AI 博物馆与联机参观设计

AI 博物馆入口为 `/ai-museum`，代码位于 `frontend/src/app/modes/ai-museum`。前端使用 Three.js 构建第一人称展厅：用户进入后可以用 WASD/方向键移动、鼠标控制视角、Shift 加速，靠近展墙时右侧导览面板显示当前展品说明。

博物馆展品按 AI 发展脉络组织，包括人工神经元、图灵测试、Dartmouth、感知机、ELIZA、专家系统、AI 寒冬、Hopfield 网络、反向传播、Q-learning、SVM、LeNet、RNN/LSTM、AlexNet、YOLO、深度强化学习、Transformer、LLM、RLHF、GRPO、多模态与智能体 workflows 等。每个展品包含年份、标题、副标题、说明、要点、标签、来源、颜色主题和 3D 位置。

前端设计重点：

能力	实现方式
第一人称漫游	Three.js <code>PointerLockControls</code> ，锁定鼠标后移动相机
展厅结构	使用长廊、墙面、地面时间线、年份标记和展墙卡片组织空间
展品内容	将展品定义为结构化 <code>MuseumExhibit</code> 数组，便于维护和扩展
动态展品	根据展品类型生成不同 3D artifact，例如注意力环、token 方块、强化学习交互符号等
导览面板	每帧根据相机位置寻找最近展品，距离足够近时显示说明
性能控制	限制展厅宽长、相机边界、像素比和动画对象数量，避免 3D 场景过重

联机能力由 Spring WebSocket 实现，后端代码位于 `MuseumPresenceHandler.java`，注册路径为 `/api/museum/presence`。用户进入博物馆后，前端建立 WebSocket 连接，并定期发送当前位置与朝向：

```
Browser A/B/C
-> ws://host/api/museum/presence?token=...
-> Spring MuseumPresenceHandler
-> room 分配、join/pose/leave 广播
-> 其他浏览器渲染远程参观者 avatar
```

联机协议：

消息类型	方向	说明
welcome	服务端 -> 当前用户	返回自己的 <code>selfId</code> 、房间号、房间人数上限和当前参与者
join	服务端 -> 房间其他用户	新用户进入房间
pose	双向	客户端发送位置，服务端广播给其他用户
leave	服务端 -> 房间其他用户	用户断开连接，移除 avatar

后端联机设计：

- 每个房间最多 8 人，控制同一展厅的远程 avatar 数量。
- 登录用户通过 JWT 识别显示名，未登录用户以“游客 N”身份进入。
- 服务端保存 `Participant` 的房间、颜色、坐标、朝向和更新时间。

- 坐标在后端做 clamp，防止异常客户端发送离谱位置。
- 房间无人时自动移除，避免长期占用内存。

5.10 LLM 浮标设计

LLM 浮标位于 `frontend/src/app/shared/llm`，后端位于 `backend/spring/src/main/java/com/deepvision/studio/llm`。

前端能力：

- 固定在页面右下角，可展开/收起。
- 支持普通问答和“传入页面上下文”两种模式。
- A 模式提供专用 prompt，如解释当前层、卷积核差异、输出 shape、答辩总结。
- 支持传入文本上下文和最多 4 张图像 URL。
- 前端做轻量 Markdown 渲染，展示列表、代码和加粗文本。

后端接口：

方法	路径	说明
POST <code>/api/llm/chat</code>	普通 JSON 聊天响应	
POST <code>/api/llm/chat/stream</code>	SSE 流式聊天响应	

后端通过 `LlmChatClient` 对接火山方舟兼容接口，配置项包括：

- `ARK_BASE_URL`
- `ARK_API_KEY`
- `ARK_MODEL`
- `ARK_CONNECT_TIMEOUT_SECONDS`
- `ARK_READ_TIMEOUT_SECONDS`

5.11 帮助文档浮标设计

帮助文档浮标位于 `frontend/src/app/shared/teaching`。其作用是降低深度学习术语门槛：

- 页面中重要术语通过 `TeachingTermDirective` 标注。
- 浮标开启后，用户可看到术语提示，并跳转 `/teaching` 教学文档。
- 浮标不改变主页面路由和实验状态。

5.12 页面美化与交互一致性

成员 A 负责整体网站视觉风格和 A 模式界面美化，主要原则：

- 顶栏统一显示当前模式、用户状态和关键操作。
- A 模式采用工作台式布局。
- 面板密度较高，但用标题、标签、状态条和弹窗区分信息层级。
- 历史记录、样本选择、卷积核对比、图片查看器均使用浮层/抽屉，保持主画布不频繁跳页。
- AI 博物馆采用沉浸式全屏布局，与工作台页面区分开，使首页既有实验入口也有展示型入口。
- LLM 浮标和帮助浮标固定在右下角，但错开位置，避免互相遮挡。

AI 辅助主要用于局部 CSS 草稿、按钮文案备选和空状态提示；最终布局、功能取舍和与实际数据绑定由人工完成。

5.13 Docker 部署方案

部署方案使用 `docker-compose.yml` 编排三个服务：

服务	说明	端口
frontend	Angular production build, 由 Nginx 托管	宿主机 4200 -> 容器 80
spring-backend	Spring Boot 后端, 连接 H2 文件库, 代理 Python 和 LLM	容器内部
python-forward	Flask + NumPy 前向计算服务	容器内部 5000

关键环境变量：

变量	作用
DEEPVISION_JWT_SECRET	JWT 密钥
DEEPVISION_FORWARD_BASE_URL	Spring 访问 Python forward 服务地址
DEEPVISION_DB_URL	H2 数据库地址
DEEPVISION_UPLOAD_ROOT	上传文件目录
ARK_API_KEY	LLM 服务 API Key
ARK_MODEL	默认大模型

数据卷：

- `spring-data`：H2 数据库。
- `spring-uploads`：A 模式历史记录预览图。
- `spring-datasets`：训练/数据集资源。
- `spring-training-jobs`：训练任务输出。

本地启动：

```
docker compose up --build
```

访问：

```
http://localhost:4200
http://localhost:4200/api/health
```

前端 Nginx 同时代理 `/swagger-ui/**` 和 `/v3/api-docs/**` 到 Spring，容器环境下 Swagger UI 可通过 `http://localhost:4200/swagger-ui/index.html` 访问。

云端部署由成员 A 人工完成，无 AI 参与。云服务器系统为 Ubuntu 22.04，公网入口为 `http://1.117.223.242/`，对外开放 HTTP 80 端口；前端 Nginx 容器监听容器内 80，并将 `/api/**`、`/swagger-ui/**`、`/v3/api-docs/**` 等路径代理到 Docker 网络内的 `spring-backend:8080`。Spring 后端通过 Docker 服务名访问 `python-forward:5000`，H2 数据库、上传文件、训练数据集和训练任务输出通过命名数据卷持久化保存。

5.14 成员 A 攻克的主要难点

难点	背景	解决方案	体现的工程能力
A 模式既要可视化又要真实计算	如果计算和渲染都放在前端，计算边界和结果来源不清晰	Python NumPy 执行图计算，Spring 只做代理，前端渲染真实结果	架构拆分、跨语言服务集成
图像张量过大导致页面卡顿	原图或中间特征图可能很大	前端设置预览尺寸上限，Python 返回可视化摘要，真实 tensor 与预览分离	性能优化、渲染控制
早期特征图渲染方式卡顿	初版为了展示卷积效果，曾把卷积输出拆成大量小块 DOM 节点逐个渲染，再在页面上拼成图像；当图片尺寸稍大时会生成数百甚至上千个 HTML 元素，浏览器布局和重绘压力很大	改为先拿到完整计算结果，再用 Canvas/ImageData 一次性合成为图片或 Data URL，页面只渲染一个 <code>img</code> 或少量通道预览节点	前端性能分析、渲染模型优化
网络层 shape 易出错	stride/padding/kernel/dilation 组合容易得到非法输出	Python 后端统一校验并返回 <code>validationIssues</code> ，前端高亮错误字段	数据校验、错误反馈
3D 网络展示信息不足	只画层结构无法说明网络运行过程	将 <code>layerSnapshots</code> 、 <code>shape</code> 、特征图、Top-K 绑定进 3D 场景	可视化抽象能力
AI 博物馆需要兼顾展示和实时联机	静态时间线缺乏沉浸感，但 3D + WebSocket 容易让状态管理复杂	前端用结构化展品数组驱动 Three.js 场景，后端用房间状态管理 <code>join/pose/leave</code> ，客户端只渲染远程 <code>avatar</code>	3D 交互设计、实时通信
历史记录要保存复杂状态	网络结构、输入、输出、预览图不是固定表结构	元数据结构化入表，完整快照以 JSON 保存，图片单独落盘	数据建模、持久化设计
LLM 不能直接暴露 Key 到前端	API Key 放前端有泄漏风险	Spring 后端代理 LLM，并支持 SSE	安全意识、后端接口设计
Docker 中服务地址变化	本地 Python 是 <code>127.0.0.1:5000</code> ，容器内需走服务名	通过 <code>DEEPVISION_FORWARD_BASE_URL</code> 区分环境	部署配置能力

5.15 成员 A 的 AI 使用复盘

5.15.1 总体边界

类型	人工负责	AI 辅助	最终责任
需求与模式设计	确定 A/B/C/D/E 平台结构，决定 A 模式做成 “可编辑网络 + 真实 forward + 3D 展示”	辅助整理功能清单、生成可选页面文案	人工筛选功能，删除超出课程规模的设计
架构拆分	决定 Angular、Spring、Python forward 三服务拆分，确定 Spring 作为统一 API 入口	辅助比较前后端通信方式和 Docker 说明表达	人工根据本地运行、部署和代码维护成本做取舍
数据结构	定义 layers、connections、inputTensor、ForwardPassResult、历史记录 snapshot 的核心字段	辅助补全 TypeScript/Java DTO 样板	人工调试前后端字段一致性，修正 shape、tensor、preview 数据
业务代码	编写和整合 A 模式状态流、保存回溯、登录注册联动、LLM 上下文、3D 快照传递	辅助生成局部函数草稿、表单代码、错误处理模板	人工逐段合并、运行、调试，并按项目结构重命名和拆分
算法实现	明确要支持 Conv/Pool/Dense/Activation/Dropout/Output，定义每层输入输出	辅助提供 NumPy 向量化卷积、softmax、统计值等参考写法	人工校验输出 shape、调通与前端可视化的数据契约
UI 与样式	确定工作台三栏布局、右下角浮标、历史记录抽屉和 3D 窗口入口	辅助生成 CSS 草稿、空状态和按钮文案	人工根据真实页面密度、中文长度和交互状态反复调整
文档整理	确定文档结构、接口表、工程难点、分工表达	辅助语言组织和表格整理	人工核对代码路径、删除夸大表述

5.15.2 重点业务代码分工

模块	人工设计与决策	AI 参与方式	人工修改/落地点
ModeAPageComponent	决定 A 模式页面状态：网络层、连接、样本、当前输入、forward 结果、记录抽屉、弹窗状态统一放在页面组件中管理	AI 辅助生成部分 getter、表单绑定、弹窗结构和错误提示草稿	人工把草稿接入真实 SimEngine、ForwardBackendService、ForwardRecordService，并处理自动计算、防抖、取消和回溯逻辑
A 模式 forward 请求	决定前端只提交抽象网络结构和输入张量，不把计算逻辑放在 DOM 渲染层	AI 辅助生成请求体类型和 service 方法	人工确定 /api/forward 契约，并让 Spring 代理到 Python，避免前端直接访问 Python 服务
Python forward 引擎	决定后端必须返回每层 layerResults、shapePath、validationIssues、stats，便于前端解释	AI 辅助提供 NumPy sliding_window_view、tensordot、softmax、Top-K 等实现参考	人工修正层类型、参数字段、错误信息和前端需要的可视化结构

模块	人工设计与决策	AI 参与方式	人工修改/落地点
特征图渲染优化	人工发现初版大量 DOM 小块拼图导致严重卡顿，并决定改成 Canvas/ImageData 一次性生成图片	AI 辅助确认浏览器 DOM 数量、重排重绘和 Canvas 渲染的性能差异	人工改造 <code>tensorToImageDataUrl</code> 、 <code>grayValuesToImageDataUrl</code> 、通道预览缓存等前端逻辑
3D 网络显示	决定通过快照传递 A 模式结果，3D 页面只负责展示，不重新计算	AI 辅助 Three.js 几何体、材质、OrbitControls 写法	人工设计层类型到几何体的映射、 <code>shape</code> 缩放策略、选中/聚焦/传播粒子交互
AI 博物馆	人工决定做成第一人称时间长廊，设计展品顺序、空间布局、移动方式和联机参观目标	AI 辅助生成部分展品说明、Three.js 展品 <code>artifact</code> 草稿和 WebSocket 消息样板	人工整合 <code>AiMuseumPageComponent</code> 、 <code>MuseumPresenceHandler</code> ，调试 <code>pointer lock</code> 、位置广播、房间人数和 <code>avatar</code> 显示
登录注册与历史记录	决定保存记录必须绑定用户，历史记录只返回当前用户数据	AI 辅助生成 <code>Controller/DTO/Repository</code> 样板	人工补充 <code>BCrypt</code> 、 <code>JWT</code> 、 <code>Principal</code> 取用户、 <code>Data URL</code> 图片落盘和 <code>/uploads/**</code> 映射
LLM 浮标	决定 LLM 不是单独页面，而是作为全局浮标接入 A 模式上下文	AI 辅助 <code>prompt</code> 、 <code>Markdown</code> 渲染和 <code>SSE</code> 接收草稿	人工控制上下文摘要、图片数量、流式异常处理和后端 <code>API Key</code> 代理
Docker 部署	决定拆为前端、Spring、Python <code>forward</code> 三容器，并使用卷保存 <code>H2</code> /上传文件	AI 辅助命令说明和环境变量说明	人工调通容器内服务名、端口映射和数据卷

5.15.3 典型 AI 辅助但人工主导的迭代案例

以 A 模式卷积结果渲染为例，初版思路是把每个卷积输出单元都当成页面元素渲染，再通过大量小块拼接出特征图。这个方案在小样本下能直观看到“每个格子”的值，但图片稍大时会创建大量 DOM 节点，导致页面滚动、刷新和参数调整都变慢。

后续重构时，渲染方案改为保留张量数组，展示阶段一次性写入 Canvas，再转换成图片地址交给 `<img>` 渲染；多通道特征图默认只展示前几个通道，完整通道通过弹窗查看。

6. 成员 B：李子涵开发内容

6.1 负责范围概述

成员 B 负责 B 模式从数据准备到训练后分析的完整业务链。该模块不是只在前端模拟曲线，而是由 Spring 创建训练任务，调用 `PyTorch worker` 读取真实数据、执行训练与测试，再把结构化指标推送回 `Angular`。

模块	主要代码路径	具体内容
B 模式训练工作台	<code>frontend/src/app/modes/mode-b/</code>	数据集选择、数据划分、网络编辑、超参数、训练控制、指标曲线、测试结果、反向传播教学面板和 <code>checkpoint</code> 操作

模块	主要代码路径	具体内容
前端训练运行时	<code>frontend/src/app/shared/training/training-runtime.service.ts</code>	REST 请求、训练 WebSocket、状态流、日志、曲线历史、反向传播快照、checkpoint 测试和单样本推理
前端数据集服务	<code>frontend/src/app/shared/training/training-dataset-api.service.ts</code>	数据集列表、详情、Multipart 上传、删除、私有预览 URL 归一化
训练协作客户端	<code>frontend/src/app/shared/training/training-collaboration.service.ts</code>	房间连接、在线成员、消息历史、流式机器人消息和房间列表
实验对比页面	<code>frontend/src/app/modes/mode-b/experiment-compare/</code>	按数据集聚合历史训练，展示指标、曲线、网络结构、层参数与 3D 入口
单样本推理页面	<code>frontend/src/app/modes/mode-b/single-inference/</code>	选择已完成 checkpoint、加载样本、执行推理、播放逐层激活
训练协作页面	<code>frontend/src/app/modes/mode-b/training-collaboration/</code>	独立窗口聊天室、训练状态、进度、日志、成员列表和 @智能助手
Spring 训练域	<code>backend/spring/src/main/java/com/deepvision/studio/training/</code>	数据集、任务、checkpoint、REST DTO、训练流 WebSocket 和协作房间
PyTorch worker	<code>backend/python-training/training_worker.py</code>	数据加载、动态建模、训练/验证/测试、梯度统计、checkpoint 与逐层激活
数据集准备脚本	<code>backend/spring/scripts/download_builtin_datasets.py</code>	下载和整理内置数据集文件，生成 Spring 与 Python 共用的目录结构
数据与部署配置	<code>backend/spring/src/main/resources/application.yml</code> , <code>docker-compose.yml</code> , <code>backend/spring/Dockerfile</code>	H2、数据集目录、任务目录、Python 路径、上传大小和持久化卷

6.2 B 模式前端结构与状态管理

B 模式主组件为 `ModeBPageComponent` 。页面把训练配置、运行状态和训练后操作组织在同一工作台

页面区域	主要交互
左侧训练数据	浏览内置数据集和当前用户上传的数据集，导入或删除数据集，选择预设任务，打开实验对比、单样本推理和聊天室
数据划分	编辑 train/validation/test 百分比，前端即时检查范围和总和
网络结构编辑器	选择 MLP、CNN、Residual CNN、CSV MLP、二分类和回归模板；增加、删除、启用或修改网络层
训练超参数	配置 batch size、epoch、学习率、优化器、scheduler、衰减和损失函数

页面区域	主要交互
当前训练状态	显示 epoch、batch、损失、准确率、学习率、梯度范数、耗时、ETA，并提供暂停、继续、停止和重置
反向传播教学面板	以网络节点展示每层梯度与更新量，点击节点查看 <code>grad_norm</code> 、 <code>update_norm</code> 曲线和真实梯度直方图
指标与结果	展示损失、准确率、学习率、梯度曲线、权重概览、测试样本预测和 checkpoint

页面状态主要来自两个来源：

- 配置态保存在 `ModeBPageComponent`，包括当前数据集、网络层、连接关系、数据划分和超参数。
- 运行态集中在根级 `TrainingRuntimeService` 的 `BehaviorSubject` 中，包括 `state$`、`history$`、`logs$`、`testResult$` 和 `backprop$`。

根级服务使用户在 Angular 单页应用内离开 B 模式再返回时，仍可读取同一训练任务的状态和 WebSocket 连接。用户身份变化时，服务会关闭旧连接并清空任务、日志、测试结果和反向传播快照，避免不同账号之间复用前端状态。当前活动任务本身保存在 Spring 进程内存中，因此浏览器整页刷新或 Spring 重启后的任务恢复，不等同于 checkpoint 的持久化恢复。

训练开始前，页面执行两类校验：

- 数据校验：是否有标签、划分比例是否为 100%、训练集比例是否大于 0。
- 模型校验：输入层和输出层是否存在、输出单元数是否匹配类别数、表格数据是否误用了卷积/池化/残差层、回归任务是否使用单输出与 MSE、逐层 shape 是否有效。

`startTraining()` 只有在校验通过后才调用后端；任务创建成功后页面平滑滚动到“当前训练状态”，训练结束后提示用户继续尝试单样本推理。

6.3 数据集管理与数据库设计

6.3.1 内置数据集

Spring 启动时由 `TrainingDatasetService.registerBuiltinDatasets()` 注册数据集元信息。当前代码包含：

数据集 ID	类型	任务
<code>mnist-1000</code>	灰度图像	10 类手写数字分类
<code>cifar10-500</code>	RGB 图像	CIFAR-10 全量分类
<code>cifar10-5000</code>	RGB 图像	CIFAR-10 5000 张课堂快速训练
<code>iris</code>	CSV 表格	3 类鸢尾花分类
<code>points-2d</code>	二维点	二分类
<code>house-price-regression</code>	合成表格	单输出回归

元信息会写入 H2，但真实图片和 CSV 位于 `datasets/builtin/**`。图片预览优先读取本地各类别样本；若资源尚未准备完成，则使用后端生成的轻量 SVG 占位预览。

6.3.2 用户上传与校验

上传入口为 `POST /api/training/datasets/imports`，使用 `Multipart` 接收文件。上传必须登录，保存后的 `owner_username` 用于列表、详情、文件读取和删除时的所有权检查。

上传类型	文件约定	主要校验
CSV	单个 <code>.csv</code> ；用户必须指定标签列和类别数	至少 10 行、表头与每行列数一致、标签列存在、至少 2 个实际类别、配置类别数不能小于实际类别数、至少一个可用特征
ZIP 图片集	单个 <code>.zip</code> ；图片放在类别目录下，如 <code>cat/001.jpg</code>	路径不能越界、忽略 <code>__MACOSX</code> 等系统文件、最多 5000 张、图片可解码、至少 2 类且每类至少 2 张
多图片	多个常见图片文件；文件名以类别开头，如 <code>cat_001.jpg</code>	不能混入 CSV/ZIP、图片可解码、类别与每类样本数满足要求

CSV worker 会把数值列直接转为浮点数，把非数值特征做简单 one-hot 编码，并忽略常见 ID/姓名列。标签列名和用户配置类别数分别写入：

```
datasets/upload/<datasetId>/label-column.txt
datasets/upload/<datasetId>/class-count.txt
```

当前上传 CSV 实现面向分类任务，因此要求至少两个类别；回归演示使用内置 `house-price-regression` 数据集。

6.3.3 training_datasets 实体

实体定义在 `TrainingDataset.java`：

字段组	字段	用途
标识与归属	<code>id</code> 、 <code>source</code> 、 <code>kind</code> 、 <code>ownerUsername</code>	区分内置/上传、图像/表格/点数据，并隔离用户数据
基本统计	<code>name</code> 、 <code>description</code> 、 <code>sampleCount</code> 、 <code>classCount</code> 、 <code>inputShape</code>	数据集列表和结构校验
训练建议	<code>recommendedSplit</code> 、 <code>trainRatio</code> 、 <code>valRatio</code> 、 <code>testRatio</code> 、 <code>hasLabels</code>	初始化数据划分和训练可用性
JSON 元信息	<code>labelsJson</code> 、 <code>labelDistributionJson</code> 、 <code>imagePreviewJson</code> 、 <code>tablePreviewJson</code> 、 <code>pointPreviewJson</code> 、 <code>warningsJson</code>	保持不同数据类型的预览结构可扩展
时间	<code>createdAt</code> 、 <code>updatedAt</code>	记录数据集元信息生命周期

删除操作只允许删除当前用户自己的上传数据集；内置数据集会被后端拒绝。删除时先递归删除 `datasets/upload/<datasetId>`，再删除 H2 记录。

6.4 网络模板、维度校验与真实模型构建

B 模式复用 `SimEngine.templates()` 和统一的 `NetworkLayer` 数据结构。当前支持：

- 图像输入和 CSV 向量输入。

- Conv2D、Pool2D、ResidualBlock、Flatten、Dense、Activation、Dropout 和 Output。
- CSV MLP、二分类 MLP、回归 MLP、普通 MLP、经典 CNN、Residual CNN 等模板。

前端使用 `SimEngine.inferLayerOutputShape()` 推导每层 shape。对于残差块，除卷积主分支输出外，还检查 shortcut 是否需要 1x1 projection；如果关闭投影后空间尺寸或通道数不同，Python `ResidualBlock.forward()` 会在相加前比较 shape，并返回明确的维度不匹配错误。

Python worker 的 `build_model()` 把前端层描述映射为 PyTorch 模块：

前端层	PyTorch 实现
conv2d	<code>nn.LazyConv2d</code> ，随后按配置附加激活函数
pool2d	<code>nn.MaxPool2d</code> 或 <code>nn.AvgPool2d</code>
residual	两层卷积、激活和可选 1x1 <code>LazyConv2d</code> shortcut
flatten	自定义 <code>AutoFlatten</code>
dense	<code>nn.LazyLinear</code> 与可选激活
dropout	<code>nn.Dropout</code>
output	<code>nn.LazyLinear</code> ；交叉熵模式下保留 logits，不额外执行 softmax

`LazyConv2d` 和 `LazyLinear` 允许 worker 根据第一条真实样本初始化输入维度，但不会替代前端的数据类型检查。例如 CSV 样本是 `[featureCount]` 向量，不能直接送入卷积层。

训练任务还支持：

- 损失：交叉熵、二元交叉熵配置和 MSE 回归。
- 优化器：Adam、AdamW、RMSProp、SGD、Momentum、Nesterov、Adagrad、Adadelta。
- 调度器：无调度、StepLR、CosineAnnealingLR。

6.5 Spring 训练任务编排

训练入口为 `POST /api/training/start`。请求结构由 `TrainingDtos.StartTrainingRequest` 定义：

字段	内容
datasetId	内置或当前用户可见的数据集 ID
split	train、val、test 比例
layers	完整网络层 JSON
connections	前端连接关系
config	batch size、epoch、学习率、优化器、scheduler、衰减和损失函数

`TrainingJobService.start()` 的执行流程：

校验登录用户与数据集可见性

- > 校验监督标签和数据划分
- > 计算 `totalEpochs / totalBatches`
- > 生成 `jobId` 和模型 SHA-256 signature
- > 创建 `training-jobs/<jobId>/`
- > 写入 `request.json` 与 `control.json`
- > `ProcessBuilder` 启动 `training_worker.py`
- > 读取合并后的 `stdout/stderr`
- > 解析逐行 JSON 并更新任务状态

每个任务目录包含：

文件	作用
<code>request.json</code>	数据集路径、网络结构、连接、超参数、划分、checkpoint 路径和模型签名
<code>control.json</code>	Spring 写入 <code>running</code> 、 <code>paused</code> 或 <code>stopped</code> ，Python 每个 batch 检查
<code>checkpoint.pt</code>	训练结束后由 PyTorch 保存的模型状态

暂停和继续不是终止并重建进程，而是通过 `control.json` 让 worker 在 batch 边界等待；停止会写入停止状态并销毁 Python 进程；重置会清空当前内存指标并重新启动同一任务。

6.6 WebSocket 指标流与反向传播观察

训练 worker 使用 `emit()` 向标准输出打印单行 JSON。Spring 只解析以 JSON 表示的训练事件，非结构化诊断文本不会作为指标广播。

事件类型	主要字段	前端用途
<code>metric</code>	<code>epoch</code> 、 <code>batch</code> 、 <code>loss</code> 、 <code>valLoss</code> 、 <code>accuracy</code> 、 <code>valAccuracy</code> 、 <code>lr</code> 、耗时、 <code>ETA</code> 、 <code>gradientNorm</code> 、 <code>weightMean</code> 、 <code>weightStd</code>	当前状态和四条历史曲线
<code>backprop</code>	<code>phase</code> 、全局梯度/更新范数、每层统计、预测解释	反向传播网络节点、曲线、直方图和诊断
<code>test_result</code>	测试损失、测试准确率、样本预测	测试集效果和自动保存 checkpoint
<code>control</code>	<code>running/paused/stopped/completed</code>	同步任务生命周期
<code>error</code>	原始异常文本	训练日志和停止状态

`TrainingStreamHandler` 从查询参数读取 `jobId` 和 JWT，调用 `requireOwnedJob()` 校验任务所有者。连接建立后，Spring 会先补发内存中最近最多 240 条事件，再继续推送新事件，因此同一 SPA 中临时离开页面再返回时可以恢复近期视图。

PyTorch 反向传播数据来自真实 batch：

1. 前向计算后发出 `forward` 事件。

- 2. 计算损失后发出 `loss` 事件。
- 3. `loss.backward()` 后统计每层 `gradNorm`、`gradMean`、`gradMax`、14 桶梯度直方图、`weightNorm` 和参数量。
- 4. `optimizer.step()` 前后比较参数快照，计算 `updateNorm`。
- 5. 每个 epoch 验证完成后补充验证阶段快照。

为了控制传输量，完整阶段事件主要采集每个 epoch 的第一个 batch，epoch 结束后再推送汇总指标。前端为每层保留最近 80 个反向传播采样点，点击节点后在弹窗中展示：

- `grad_norm` 随采样 step 变化。
- `update_norm` 随采样 step 变化。
- 梯度值区间与参数数量组成的直方图。
- 梯度过小、偏小、正常、偏大、过大的分级提示。

训练主图中的损失、准确率、学习率和梯度曲线来自真实 `metric` 事件。权重分布概览没有传输完整参数数组，而是根据 worker 返回的真实 `weightMean` 和 `weightStd` 生成近似分布，用于控制 WebSocket 消息体；弹窗中的梯度直方图则由真实梯度张量计算。

6.7 Checkpoint 与实验对比

6.7.1 权重与元信息保存

Python 通过 `torch.save()` 把以下内容写入 `checkpoint.pt`：

- `modelStateDict`
- `layers`
- `datasetId`
- `config`
- `classCount`
- `modelSignature`

Spring 收到 `test_result` 后自动调用 `saveCheckpoint()`，将可检索元信息写入 H2 的 `training_checkpoints`。数据库不直接保存 `.pt` 二进制文件，只保存 `checkpointPath`。

元信息	对应字段
用户与任务	<code>user_id</code> 、 <code>jobId</code> 、 <code>name</code> 、 <code>createdAt</code>
数据集	<code>datasetId</code> 、 <code>datasetName</code>
模型标识	<code>modelSignature</code> 、 <code>checkpointPath</code> 、 <code>networkDescription</code>
可重建配置	<code>layersJson</code> 、 <code>configJson</code> 、 <code>splitJson</code>
训练过程	<code>metricHistoryJson</code> 、 <code>status</code> 、 <code>epoch</code> 、 <code>totalEpochs</code>
最终结果	<code>train/val/test loss</code> 、 <code>accuracy</code> 、 <code>测试样本数</code> 、 <code>testResultJson</code>

checkpoint 查询通过 `TrainingCheckpointRepository` 限定当前用户名。测试、样本列表和单样本推理同样使用 `findByIdAndUserUsername()`，避免用户通过修改 ID 读取其他账号的模型。

6.7.2 实验对比页面

实验对比路由为 `/training/experiments`。页面先加载当前用户的 `checkpoint`，再按 `datasetId` 聚合：

- 数据集训练次数、最佳测试准确率和最近训练时间。
- 每次训练的 train/validation/test 指标。
- 优化器、学习率、batch、epoch、损失函数和数据划分。
- 网络结构文本、可点击网络节点和完整层参数。
- 损失、准确率、学习率和梯度范数历史曲线。
- 通过共享 `NetworkOverviewComponent` 展示网络，并把结构写入本地存储后打开 `/network-3d`。

页面对 `stopped`、epoch 未跑满和缺少历史曲线的旧记录分别显示状态说明，避免把不完整记录当作正常完成实验比较。

6.8 单样本推理与逐层激活

单样本推理路由为 `/training/inference`，只列出未停止且 `epoch >= totalEpochs` 的 checkpoint。

运行流程：

选择 checkpoint

-> GET `/api/training/checkpoints/{id}/samples?limit=72`

-> Python 只加载限定数量的预览样本

-> 用户选择一条图片或表格记录

-> POST `/api/training/checkpoints/{id}/infer`

-> Python 加载 `checkpoint.pt` 和原训练结构

-> 对该样本逐模块前向传播

-> 返回 `prediction + activations`

-> Angular 按层播放激活网络

图片样本返回受控资源 URL；表格样本返回原始表头、原始行、编码后特征预览和特征名。页面默认展示一部分原始字段，点击后用弹窗查看完整行，避免把 one-hot 后的向量误当作用户原始 CSV。

每层激活包含：

字段	说明
<code>layerId</code> 、 <code>layerName</code> 、 <code>layerType</code> 、 <code>order</code>	与原网络结构对应
<code>shape</code>	去掉 batch 维后的输出形状
<code>stats</code>	min、max、mean、nonZeroRatio
<code>preview</code>	图像层最多 6 个下采样通道；向量层最多 40 个值
<code>topValues</code>	绝对值最大的若干激活位置

前端按约 950 ms 自动切换活动层，也允许用户点击节点停止自动播放并查看指定层。checkpoint 列表使用滚动选择，鼠标悬停时在列表右侧弹出超参数、数据划分、指标和网络层摘要。

6.9 多人协作聊天室

训练协作页面以新浏览器标签打开，不改变原 B 模式页面。B 模式支持两种入口：

- 基于当前训练任务创建聊天室。
- 读取现有房间列表并加入已经存在的聊天室；不存在的房间不会跳转。

聊天室使用两个 WebSocket:

WebSocket	职责
/api/training/collaboration	房间创建/加入、聊天消息、最近 60 条历史、在线成员和系统通知
/api/training/collaboration/stream	对已经加入房间的 clientId 推送训练指标、反向传播、测试结果和日志

TrainingCollaborationHandler 在最后一个会话离开时删除内存房间，因此空房间自动销毁。房间只保存内存消息，不写入 H2。

用户输入 @ 时，前端显示“智能助手”选择框；发送以 @智能助手 开头的问题后，Spring 读取当前训练状态和最近聊天上下文，调用 LlmChatClient.stream()。机器人先广播空的流式消息，再用 chat_update 持续替换文本，所有房间成员看到同一回复过程。

聊天室训练区复用 TrainingRuntimeService.observeCollaborationJob()，展示 epoch、batch、loss、validation loss、accuracy、validation accuracy、学习率、梯度范数、进度、耗时、ETA 和训练日志。协作观察流是只读的，任务暂停、停止等控制仍留在任务拥有者的 B 模式页面。

6.10 REST 接口与主要数据结构

6.10.1 主要 REST 接口

方法	路径	作用	用户边界
GET	/api/training/datasets	内置数据集加当前用户上传数据集	未登录只返回内置数据
GET	/api/training/datasets/{datasetId}	数据集详情	上传数据集校验 owner
POST	/api/training/datasets/imports	导入 CSV/ZIP/图片	必须登录
DELETE	/api/training/datasets/{datasetId}	删除上传数据集	仅 owner，内置不可删
GET	/api/training/datasets/{datasetId}/files/**	读取私有上传文件	校验 owner 与安全相对路径
POST	/api/training/start	创建真实训练任务	必须登录
GET	/api/training/{jobId}/status	查询任务状态	仅任务 owner
POST	/api/training/{jobId}/pause	resume	stop
GET	/api/training/checkpoints	查询当前用户 checkpoint	必须登录，可按 datasetId 过滤
POST	/api/training/checkpoints/{id}/test	重新运行测试集	仅 checkpoint owner
GET	/api/training/checkpoints/{id}/samples	获取有限样本预览	仅 checkpoint owner，且训练完成

方法	路径	作用	用户边界
POST	/api/training/checkpoints/{id}/infer	单样本推理和逐层激活	仅 checkpoint owner，且训练完成
GET	/api/training/collaboration/rooms	查询当前内存房间	用于加入已有房间

SecurityConfig 当前对 /api/training/** 做统一放行，以便 WebSocket 握手和部分公开查询进入 Controller；需要身份的业务操作在 TrainingDatasetService 和 TrainingJobService 内再次检查 Principal、JWT subject、owner 或 checkpoint 用户。训练拥有者流的 WebSocket 还会单独校验 token。

6.10.2 前后端共享数据含义

类型	定义位置	用途
NetworkLayer、TrainingConfig、TrainingDatasetDetail	frontend/src/app/shared/simulation/sim-models.ts	网络、训练配置和数据集详情
TrainingRuntimeState、MetricPoint	同上	当前训练状态和历史曲线点
TrainingBackpropSnapshot、BackpropLayerStat	training-runtime.service.ts	真实反向传播事件
TrainingCheckpointSummary	training-runtime.service.ts / TrainingDtos.java	checkpoint 列表、实验对比和推理选择
InferenceSampleItem、SingleInferenceResult	同上	样本预览、预测和逐层激活
CollaborationMessage、CollaborationRoomSummary	training-collaboration.service.ts / TrainingDtos.java	聊天、房间和在线状态

6.11 运行与部署

本地运行 B 模式需要 Angular 和 Spring。PyTorch worker 由 Spring 自动启动，不需要再手动启动一个训练 HTTP 服务，但必须配置可用 Python 环境：

```
$env:DEEPVISION_TRAINING_PYTHON="C:\path\to\python.exe"
$env:DEEPVISION_TRAINING_WORKER_SCRIPT="..\python-training\training_worker.py"
$env:DEEPVISION_DATASET_ROOT="..\datasets"
$env:DEEPVISION_TRAINING_JOBS_ROOT="..\training-jobs"
```

Python 依赖定义在 backend/python-training/requirements.txt，主要为 torch 和 Pillow。

Docker 部署时，backend/spring/Dockerfile 在 Spring 运行镜像中安装 CPU 版 PyTorch，并复制 backend/python-training。因此容器拓扑仍是前端、Spring、Python forward 三个容器，但 Spring 容器内部还会为每个 B 模式任务创建训练子进程。

需要持久化的卷：

卷	内容
spring-data	H2 用户、数据集元信息、checkpoint 元信息
spring-datasets	内置和上传训练数据
spring-training-jobs	request.json、control.json、checkpoint.pt 和临时 checkpoint 推理任务
spring-uploads	A 模式等其他上传资源

Nginx 对 `/api/` 开启 WebSocket Upgrade 并设置较长读取超时，同时把 `client_max_body_size` 调整为 220 MB。Spring Multipart 默认限制为单文件 200 MB、请求 220 MB。

6.12 工程难点与解决方式

6.12.1 前端网络描述与 PyTorch 模型必须一致

- 问题背景：用户在 Angular 中自由组合输入层、卷积、残差、全连接和输出层，Python 必须按同一结构训练。
- 为什么难：图像和 CSV 的张量维度不同，残差 shortcut 还要求主分支和旁路 shape 完全一致。
- 解决方式：统一 `NetworkLayer` JSON 契约；前端先做逐层 shape 和任务类型校验，worker 再用真实样本初始化 Lazy 层，并在残差相加前做运行时 shape 检查。
- 体现的工程能力：跨语言数据契约、前后端双层校验和错误定位。

6.12.2 用户数据集格式不可控

- 问题背景：课程平台允许上传 CSV、图片和 ZIP，但用户文件可能缺列、标签为空、目录错误、图片损坏或包含系统隐藏文件。
- 为什么难：校验过松会在训练阶段才报错，校验过严又会拒绝可修复的数据。
- 解决方式：上传阶段校验结构性错误；缺失值、类别不均衡和尺寸不一致作为 warnings；CSV 由用户明确选择标签列和类别数；ZIP 忽略 macOS 元数据并防止路径穿越。
- 体现的工程能力：输入边界设计、异常分级、文件安全和可用性平衡。

6.12.3 长时间训练与页面实时更新

- 问题背景：普通 HTTP 请求不适合持续返回 epoch、反向传播阶段和测试结果。
- 为什么难：需要处理任务控制、断开重连、事件顺序、用户隔离和前端状态一致性。
- 解决方式：Spring 用子进程标准输出接收 JSON 事件，训练状态保存在并发 Map，最近事件用有界队列缓存，再通过 WebSocket 推送；Angular 用 RxJS BehaviorSubject 统一驱动多个页面。
- 体现的工程能力：进程编排、事件流设计、并发容器和响应式状态管理。

6.12.4 Checkpoint 不能只保存权重

- 问题背景：只有 `.pt` 文件无法解释它使用了哪个数据集、网络和超参数，也无法在实验对比页检索。
- 为什么难：二进制权重适合文件加载，实验元信息又需要关系查询和用户隔离。
- 解决方式：采用文件与 H2 混合存储；`.pt` 保存 PyTorch 状态，H2 保存路径、结构、配置、划分、曲线和指标，并使用模型签名关联结构。
- 体现的工程能力：结构化数据与大文件的存储取舍、可追溯实验设计。

6.12.5 真实梯度可视化的数据量控制

- 问题背景：每个 batch、每一层、每个参数都传输梯度会产生很大的消息体并拖慢训练。
- 为什么难：教学界面既要显示真实信息，又不能把完整张量持续发送到浏览器。

- 解决方式：传输每层范数、均值、最大值、参数量和固定桶数直方图；阶段快照主要采样每个 epoch 的第一个 batch，曲线只保留有界历史。
- 体现的工程能力：可观测性粒度、网络开销和教学信息量之间的权衡。

6.12.6 协作房间同时承载聊天和训练观察

- 问题背景：聊天室参与者需要看到同一训练状态，但不能获得任务控制权。
- 为什么难：聊天消息和训练指标的数据频率、权限和生命周期不同。
- 解决方式：拆分聊天 WebSocket 与只读训练流 WebSocket；使用 `clientId` 确认观察者已经加入房间；最后一人离开时删除房间；机器人回答通过增量更新广播。
- 体现的工程能力：实时协作协议、权限边界和资源生命周期管理。

6.12.7 本地与云端文件路径不同

- 问题背景：本地使用 Windows 路径，Docker/云端使用 Linux 卷路径，数据集预览和 checkpoint 又必须跨组件访问。
- 为什么难：把绝对路径写死会导致迁移后图片、数据集或 worker 脚本找不到。
- 解决方式：将数据库、数据集、训练任务、Python 解释器和 worker 脚本全部抽成环境变量；前端对相对资源 URL 统一拼接 API base URL；Docker 使用命名卷固定容器路径。
- 体现的工程能力：配置外置、跨平台路径处理和部署资源同步。

6.13 人工与 AI 的分工说明

成员 B 的开发中，AI 参与了代码量较大的训练业务模块的局部实现，但没有作为独立成员或独立决策者。

工作类型	人工负责	AI 辅助方式	最终处理
需求与模块边界	确定 B 模式覆盖数据上传、真实训练、checkpoint、对比、推理和协作，而不是停留在曲线模拟	辅助拆分页面与服务清单	人工结合课程目标和现有 A 模式架构确定范围
数据结构与接口	确定 <code>NetworkLayer</code> 、训练请求、数据集元信息、checkpoint 元信息和 WebSocket 事件字段	生成部分 DTO、接口和 TypeScript interface 样板	人工统一字段、补充用户隔离并联动前后端
PyTorch 训练与可视化	确定数据加载、动态建模、训练/验证/测试、梯度范数、更新量和激活数据的采集方式	提供 PyTorch API、梯度统计和图表实现参考	人工调试维度、损失函数、残差块、CSV 编码和真实输出
WebSocket 与聊天室	设计任务拥有者流、协作旁观流、房间生命周期和机器人 mention 交互	辅助生成 Handler、客户端 service 和流式消息样板	人工修复不存在房间跳转、空房销毁、训练日志同步和账号状态问题
数据集与数据库	设计内置/上传数据集目录、H2 元信息、owner 字段和文件删除流程	辅助编写校验分支、JPA 样板和报错分析	人工处理 CSV 标签列、类别数、ZIP 隐藏文件、上传大小和云端资源路径
页面与样式	确定训练工作台、实验对比、推理和聊天室的页面结构与交互优先级	辅助生成局部 HTML/CSS、曲线和弹窗草稿	人工多轮删改开发期提示、统一交互、检查内容密度和最终效果

工作类型	人工负责	AI 辅助方式	最终处理
排错与验收	负责本地/云端联调、账号隔离、checkpoint 加载、图片预览和训练错误复现	辅助分析异常堆栈与提供排查方向	人工修改代码、重新运行并决定是否接受结果

人工主导的内容包括需求目标、模块边界、核心数据契约、训练流程、用户隔离、关键 bug 修复、部署配置和答辩取舍。AI 主要用于样板生成、局部算法与可视化参考、报错分析、样式草稿和文档整理；所有进入仓库的代码和本文档内容均由人工结合实际代码复核、删改和验收。

7. 成员 C：肖羽平开发内容

7.1 负责范围概述

成员 C 主要负责模式 C 与模式 D 两个解释类模块，并补充了它们与平台统一能力的接入工作。对应代码范围如下：

负责内容	主要路径	说明
模式 C：CNN 卷积过程解释	<code>frontend/src/app/modes/mode-c/</code>	将原始 <code>cnn-explainer</code> 重构为 Angular 原生组件，保留卷积网络总览、卷积 / ReLU / Pool 细节、Grad-CAM 和解释报告
模式 D：Transformer 注意力 / QKV 解释	<code>frontend/src/app/modes/mode-d/</code>	基于浏览器端 Transformers 运行时实现下一词预测、Top-K、注意力矩阵和 QKV 教学视图
模式 D 静态资源同步	<code>scripts/sync-transformer-explainer.ps1</code> 、 <code>docs/mode-d-assets-sync-guide.md</code>	将大模型 tokenizer、ONNX 分片和前端资源从外部源码目录同步到平台公共静态目录
平台共享能力复用	<code>frontend/src/app/shared/llm/</code> 、 <code>frontend/src/app/shared/teaching/</code> 、 <code>frontend/src/app/core/auth/</code>	为模式 C、D 接入统一顶栏、登录状态、AI 助手和教学文档入口

这部分工作的重点不是单纯嵌入第三方 demo，而是在不破坏现有 Angular 架构的前提下，把解释器重构为平台内可维护、可扩展、可与统一组件协作的教学模块。

7.2 模式 C：CNN 卷积过程解释

7.2.1 模块定位

模式 C 面向卷积神经网络基础教学，目标是在真实推理结果基础上，展示输入样例如何经过卷积层、激活层、池化层和输出层得到分类结论。与模式 A 的“可编辑网络 + 后端计算”不同，模式 C 采用浏览器端原生推理，重点放在解释视图而不是网络编辑。

7.2.2 前端结构与主要代码路径

层级	主要路径	作用
页面入口	<code>frontend/src/app/modes/mode-c/mode-c-page.component.ts</code>	负责顶栏、登录状态、AI 助手、教学入口和解释器容器装配
页面模板	<code>frontend/src/app/modes/mode-c/mode-c-page.component.html</code>	定义平台页面骨架，挂载统一头部与解释器工作区
解释器壳组件	<code>frontend/src/app/modes/mode-c/explainer/components/shell/mode-c-explainer-shell.component.ts</code>	负责样本切换、总览区域、细节面板和解释报告的页面组织
总览视图	<code>frontend/src/app/modes/mode-c/explainer/components/overview/mode-c-overview.component.ts</code>	展示各层拓扑、通道预览、层间连线、交互选中状态和局部 overlay
细节面板	<code>frontend/src/app/modes/mode-c/explainer/components/detail-panels/mode-c-detail-panel.component.ts</code>	根据层类型切换卷积、ReLU、池化、输出层等解释界面
状态服务	<code>frontend/src/app/modes/mode-c/explainer/services/mode-c-state.service.ts</code>	管理当前样本、当前层、当前通道、Grad-CAM 开关和解释文本所需状态
推理服务	<code>frontend/src/app/modes/mode-c/explainer/services/mode-c-inference.service.ts</code>	加载 tf.js 模型，执行前向推理，生成 overview / detail / Grad-CAM 所需数据
模型与资源服务	<code>frontend/src/app/modes/mode-c/explainer/services/mode-c-model.service.ts</code> 、 <code>mode-c-assets.service.ts</code>	提供模型加载、样本图片、标签和静态资源路径封装
数据结构	<code>frontend/src/app/modes/mode-c/explainer/models/mode-c.types.ts</code>	统一定义层摘要、通道预览、细节运行时、预测结果和 Grad-CAM 数据

7.2.3 运行流程

用户进入 `/mode-c`

- > `ModeCPageComponent` 挂载统一顶栏、AI 助手、教学浮标
- > `ModeCExplainerShellComponent` 初始化默认样本
- > `ModeCInferenceService` 加载 `/mode-c/cnn-explainer/assets/data/model.json`
- > 浏览器端 `tf.js` 执行推理，产出各层激活、输出概率和 Grad-CAM 中间结果
- > `ModeCStateService` 保存当前样本、层和通道状态
- > `Overview` 展示全局拓扑，`Detail Panel` 展示选中层的局部解释

其中浏览器端运行时依赖由 `frontend/public/mode-c/cnn-explainer/vendor/tf.min.js` 提供，样本、模型、标签和补充静态文件位于 `frontend/public/mode-c/cnn-explainer/assets/`。

7.2.4 核心数据结构

数据结构	定义位置	用途
ModeCNetworkLayer	mode-c.types.ts	描述 overview 中每一层的名称、类型、shape、显示顺序等元信息
ModeCLayerActivationSummary	mode-c.types.ts	记录层级摘要信息，供总览卡片和层描述使用
ModeCLayerChannelPreview	mode-c.types.ts	存放每一层各通道的预览图、取值范围和选中展示所需字段
ModeCLayerDetailRuntime	mode-c.types.ts	承载细节面板当前层的运行时数据，根据层类型分发给卷积 / ReLU / Pool / Output 视图
ModeCConvChannelExample	mode-c.types.ts	表示卷积层单个输入通道到目标输出位置的 patch、kernel、product 和中间和
ModeCGradCamResult	mode-c.types.ts	存储 Grad-CAM 热力图、叠加图和对应类别
ModeCSamplePrediction	mode-c.types.ts	表示 Top-K 预测、概率和最终分类标签

7.2.5 具体实现逻辑

1. 总览拓扑

- ModeCOverviewComponent 按 ModeCNetworkLayer[] 渲染输入层、卷积层、激活层、池化层和输出层。
- 每一层的通道图来自 ModeCLayerChannelPreview，不是静态示意图，而是推理后生成的真实激活预览。
- 组件内部维护局部交互状态，用于控制当前选中层、当前通道和是否展开细节。

2. 卷积过程解释

- ModeCInferenceService 在生成细节运行时，会截取当前输出位置对应的输入 patch、卷积核和逐元素乘积结果。
- 卷积细节不是只显示最终值，而是拆成“输入通道 -> 中间响应 -> 偏置 -> 目标输出单元”的解释链路。
- 为了保证数值与真实模型一致，卷积示例直接基于推理时提取到的张量计算，而不是使用手写演示数据。

3. ReLU 与池化解释

- ReLU 细节面板根据鼠标选中的像素位置同步显示输入值、 $\max(0, x)$ 结果和输出值。
- Pool 细节面板展示当前池化窗口覆盖的局部区域，并列参与最大值比较的候选数值。

4. 输出层与 Grad-CAM

- 输出层显示真实 Top-K 分类结果，而不是固定示意标签。
- ModeCInferenceService 基于目标卷积层特征图和输出类别梯度生成 ModeCGradCamResult，再与样本图进行叠加，形成热力图解释。
- 早期实现中 output 与 softmax 结果曾与原项目不一致，后续通过对齐 flatten 顺序、预处理方式和 logits 计算链路修正。

7.2.6 页面结构、交互设计与状态管理

模式 C 页面采用“统一顶栏 + 中央解释器工作区 + 右下角共享浮标”的结构：

- 顶栏复用平台统一头部，只保留返回首页、当前模式、登录状态及用户入口。
- 中央区域分为 overview 与 detail panel 两部分，overview 占据主要空间，detail panel 放在下方承接当前层的教学解释。
- 右下角复用 LLM AI 助手与教学问号入口，但它们不直接参与卷积推理，只消费模式 C 当前上下文。

状态流主要分为两层：

- `ModeCStateService` 管理跨组件共享的样本、层、通道、预测和 Grad-CAM 结果。
- 具体面板组件维护局部 hover、播放步进、overlay 显示等 UI 状态，避免将所有临时交互都堆积到页面根组件。

7.2.7 与平台其他模块的接入关系

模式 C 没有新增专用 Spring 业务接口，也没有使用 WebSocket。它与平台的主要接入点是：

接入项	代码路径	说明
登录状态	<code>frontend/src/app/core/auth/auth.service.ts</code>	顶栏读取当前用户信息，决定显示登录 / 注册入口还是用户状态
AI 助手	<code>frontend/src/app/shared/llm/</code>	将当前样本、当前层、预测结果等整理为上下文，调用 Spring 代理的 LLM 接口
教学入口	<code>frontend/src/app/shared/teaching/</code>	通过右下角问号浮标跳转教学文档页
样式对齐	<code>frontend/src/app/shared/components/platform-topbar.component.ts</code>	保持与模式 A、模式 B 相同的平台顶栏和页面节奏

7.3 模式 D：Transformer 注意力 / QKV 解释

7.3.1 模块定位

模式 D 面向 Transformer 基础教学，目标是在浏览器中完成最小可用的真实推理，并把“下一词预测 - 注意力矩阵 - QKV 交互解释”组织成可演示的教学视图。该模块不是完整大模型对话系统，而是围绕单条输入文本做结构化解释。

7.3.2 前端结构与主要代码路径

层级	主要路径	作用
页面入口	<code>frontend/src/app/modes/mode-d/mode-d-page.component.ts</code>	负责模式 D 页面骨架、顶栏、AI 助手和教学入口
页面模板	<code>frontend/src/app/modes/mode-d/mode-d-page.component.html</code>	组织解释器工作区和共享浮标

层级	主要路径	作用
解释器壳组件	<code>frontend/src/app/modes/mode-d/explainer/components/shell/mode-d-explainer-shell.component.ts</code>	汇总输入区、Top-K、注意力矩阵、QKV 面板和解释报告
输入面板	<code>frontend/src/app/modes/mode-d/explainer/components/input-panel/mode-d-input-panel.component.ts</code>	负责示例切换、输入文本、层 / 头选择与重新推理入口
注意力矩阵	<code>frontend/src/app/modes/mode-d/explainer/components/attention-matrix/mode-d-attention-matrix.component.ts</code>	展示单层单头注意力矩阵及当前高亮单元
QKV 面板	<code>frontend/src/app/modes/mode-d/explainer/components/qkv-panel/mode-d-qkv-panel.component.ts</code>	演示 Query / Key / Value、缩放点积和 value 汇入输出
报告面板	<code>frontend/src/app/modes/mode-d/explainer/components/report-panel/mode-d-report-panel.component.ts</code>	根据推理结果生成中文说明性解释
状态服务	<code>frontend/src/app/modes/mode-d/explainer/services/mode-d-state.service.ts</code>	管理文本、token、Top-K、注意力矩阵选中状态、QKV 视图和自动解释
推理服务	<code>frontend/src/app/modes/mode-d/explainer/services/mode-d-inference.service.ts</code>	初始化 tokenizer、加载 ONNX 会话、执行下一词推理并提取注意力数据
资源服务	<code>frontend/src/app/modes/mode-d/explainer/services/mode-d-assets.service.ts</code>	统一封装静态资源基路径 <code>/mode-d-assets</code>
数据结构	<code>frontend/src/app/modes/mode-d/explainer/models/mode-d.types.ts</code>	统一定义输入样例、token 分数、注意力摘要、QKV 教学数据和报告结构

7.3.3 运行流程

用户进入 `/mode-d`

- > `ModeDPageComponent` 挂载统一顶栏、AI 助手、教学浮标
- > `ModeDExplainerShellComponent` 初始化默认样例文本
- > `ModeDInferenceService` 加载 `tokenizer` 与 `GPT-2 ONNX` 会话
- > 浏览器端执行下一词推理并提取 `logits`、`token`、`attention` 张量
- > `ModeDStateService` 计算 `Top-K`、当前层头矩阵、高亮单元和 `QKV` 解释数据
- > 各子组件渲染输入区、注意力矩阵、`QKV` 视图和文字报告

模式 D 不通过 Spring 调用推理接口，而是把推理链路放在浏览器端完成。Spring 后端只负责平台通用的认证和 LLM 服务代理。

7.3.4 核心数据结构

数据结构	定义位置	用途
ModeDExample	mode- d.types.ts	预置文本样例、分类标签和教学提示信息
ModeDInferenceResult	mode- d.types.ts	浏览器端推理完成后的总体结果，包含 token、logits、attention 和下游展示字段
ModeDTokenScore	mode- d.types.ts	表示 Top-K 候选 token 及其概率
ModeDAttentionSummary	mode- d.types.ts	表示当前层 / 头的注意力矩阵、行列标签和高亮位置
ModeDQkvTeachingData	mode- d.types.ts	组织 Query、Key、Value、缩放点积、softmax 权重和值汇入输出的教学数据
ModeDReportSection	mode- d.types.ts	报告面板中使用的结构化中文解释段落

7.3.5 具体实现逻辑

1. 浏览器端推理链

- ModeDInferenceService 使用 @xenova/transformers 的 tokenizer 将文本编码成 token ids。
- 模型推理由 onnxruntime-web 完成。为避免单文件资源过大，ONNX 模型被拆成多个分片，同步到 frontend/public/mode-d-assets/model/ 后再在运行时合并。
- 推理结果中，logits 从 linear_output 读取；注意力矩阵从以 _attn_dropout 结尾的输出张量中提取。

2. Top-K 与注意力矩阵

- ModeDStateService 根据最后一个位置的 logits 计算下一个 token 的 Top-K 概率列表。
- 用户可切换层和头，矩阵组件只渲染当前层 / 当前头的注意力权重。
- 当前高亮单元会同步驱动下方解释文本与 QKV 面板，保证所有解释围绕同一个 query-token / key-token 对展开。

3. QKV 教学视图

- QKV 视图不是直接照搬外部页面，而是基于当前高亮的 query 与 key 索引生成教学数据。
- 面板中分别显示 Query、Key、Value 向量，展示 $Q \times K$ 、缩放、softmax 权重以及 value 汇入输出的过程。
- 为了让教学演示可读，QKV 页面默认聚焦当前选中单元，同时保留逐字段中文解释和数值条形图。

4. 自动解释报告

- 报告面板根据当前输入文本、Top-K、最强注意力对和 QKV 结果生成中文解释。
- 解释报告属于前端结构化生成，不依赖后端额外推理，以保证页面离线时仍能呈现基础教学结果。

7.3.6 资源同步与项目结构适配

模式 D 的模型资源较大，不适合长期直接作为源码目录的一部分维护，因此采用“源码与资源分离、按需同步”的方案：

资源类型	目标路径	同步方式
tokenizer 配置	frontend/public/mode-d-assets/tokenizer/	由 scripts/sync-transformer-explainer.ps1 从外部 transformer 项目复制
ONNX 模型分片	frontend/public/mode-d-assets/model/	同步脚本复制并保留浏览器端加载所需目录结构
额外元数据	frontend/public/mode-d-assets/	统一落在公共静态目录，便于 Angular 直接以 /mode-d-assets/** 访问

该方案对应的维护文档为 docs/mode-d-assets-sync-guide.md 。这样做的原因有三点：

- 减少仓库主分支对大体积二进制资源的直接耦合。
- 让模式 D 的运行依赖变成“源码 + 一次同步脚本”，便于后续迁移或替换模型。
- 让静态资源路径保持稳定，避免组件代码依赖外部工程的原始目录结构。

7.3.7 与平台其他模块的接入关系

模式 D 与模式 C 一样，没有新增专用 Spring 推理接口，也没有使用 WebSocket；但接入了平台统一能力：

接入项	代码路径	说明
登录状态	frontend/src/app/core/auth/auth.service.ts	统一显示当前用户状态和登录 / 注册入口
AI 助手	frontend/src/app/shared/llm/	基于当前文本、Top-K、最强注意力单元等上下文向 Spring 的 /api/llm/chat 发起请求
教学入口	frontend/src/app/shared/teaching/	通过右下角问号浮标进入教学文档页面
样式与页面框架	frontend/src/app/shared/components/platform-topbar.component.ts	与模式 A、模式 B 使用相同的头部和右下角浮标布局

7.4 第三方解释器迁移与适配策略

成员 C 负责的两个模块都涉及“参考现有解释器，但不能直接嵌入使用”的问题，不过两者的迁移方式不同：

模块	原始依赖形态	平台内改造方式	原因
模式 C	cnn-explainer , Svelte + 静态脚本	以 Angular 原生组件重写页面结构和交互逻辑，保留真实模型与样本资源	Svelte 组件生命周期、状态流和 DOM 组织无法直接复用到 Angular
模式 D	transformer-explainer 相关运行资源与教学思路	保留浏览器端 tokenizer / ONNX / attention 思路，重新实现 Angular 页面和状态管理	需要对齐平台样式、共享组件和教学入口，并控制资源体积与路径

这部分工作的核心不是“套壳”，而是把外部项目中有价值的教学思路拆成平台可维护的组件、服务和静态资源方案。

7.5 工程难点与解决方式

7.5.1 外部解释器与 Angular 架构不兼容

- 问题背景：模式 C 和模式 D 最初都参考了外部解释器项目，但这些项目并不是为本平台 Angular 架构设计的。
- 为什么难：如果直接 iframe 嵌入或保留原始工程，会造成样式不统一、状态无法共享、登录 / AI / 教学入口无法接入。
- 解决方式：模式 C 采用 Angular 原生重写；模式 D 保留浏览器端推理思路，但重建页面、状态服务和资源路径。
- 体现的工程能力：能够根据项目现状做架构取舍，而不是只追求“先跑起来”。

7.5.2 模式 C 解释结果必须与真实推理一致

- 问题背景：卷积解释如果只用示意数据，教学上直观，但很容易与输出类别、softmax 结果和热力图不一致。
- 为什么难：overview、卷积局部解释、输出层和 Grad-CAM 实际上共享同一套推理结果，任一环节处理顺序不一致都会导致解释失真。
- 解决方式：将所有层摘要、通道预览、卷积细节和输出层结果统一从 ModeCInferenceService 的真实推理链生成，并修正 flatten 顺序、输入预处理和 logits 计算方式。
- 体现的工程能力：能把“好看”的教学界面和“正确”的模型计算统一起来。

7.5.3 卷积层细节不能停留在静态示意图

- 问题背景：简单的层级缩略图无法说明卷积层到底如何从输入 patch 计算到某个输出单元。
- 为什么难：真实卷积涉及多输入通道、局部 patch、kernel 权重、偏置累加和输出位置映射，信息量大且容易把页面挤乱。
- 解决方式：把解释拆成 overview 与 detail panel 两层；overview 保留全局拓扑，detail panel 展示单输出位置的多通道中间响应、偏置和目标输出单元。
- 体现的工程能力：能够通过分层界面设计控制复杂度。

7.5.4 浏览器端 Transformer 推理链的可运行性

- 问题背景：模式 D 需要在浏览器中完成 tokenizer、模型加载和 attention 提取，而不是依赖后端代算。
- 为什么难：浏览器端对模型体积、WASM/ONNX 运行环境、资源路径和首轮初始化都更敏感，出错后也更难定位。
- 解决方式：将推理逻辑封装到 ModeDInferenceService，显式管理 tokenizer、session、logits、attention 的初始化状态，并把模型资源拆分到公共静态目录后由同步脚本维护。
- 体现的工程能力：具备对浏览器运行时限制、静态资源组织和推理链调试的处理能力。

7.5.5 大模型静态资源的仓库维护成本

- 问题背景：模式 D 所需 tokenizer 和 ONNX 资源体积较大，直接纳入源码目录会提高仓库维护成本。
- 为什么难：课程项目既要保证别人能复现，也要避免仓库长期被大文件拖慢。
- 解决方式：把大资源沉到 frontend/public/mode-d-assets/，通过 scripts/sync-transformer-explainer.ps1 做同步，并配套 docs/mode-d-assets-sync-guide.md 说明获取和更新方式。
- 体现的工程能力：在可运行性与可维护性之间做平衡。

7.5.6 AI 助手需要接入解释模块上下文

- 问题背景：如果右下角 AI 助手只是通用聊天框，它无法围绕当前样本、当前层或当前注意力头给出针对性解释。
- 为什么难：模式 C 与模式 D 的上下文字段完全不同，需要页面层负责提取、压缩和传递。

- 解决方式：在各自页面组件中构造 context provider，只把当前样本、预测、层信息、Top-K 或注意力焦点等必要信息传给共享 LLM 组件。
- 体现的工程能力：能够在复用共享组件的同时做页面级定制接入。

7.6 人工与 AI 的分工说明

成员 C 的工作中，AI 参与了部分高代码量模块的局部实现，但需求边界、数据契约、最终代码整合和结果验收由人工负责。

类型	人工负责	AI 参与方式	最终落点
模块边界与目标	确定模式 C 做 CNN 解释、模式 D 做 Transformer 注意力 / QKV 教学，不把它们做成独立外部 demo	辅助列出可能功能项和迁移思路	人工根据课程目标和平台结构筛选功能范围
组件与服务结构	确定页面组件、shell、overview、detail panel、state service、inference service 的拆分方式	辅助生成部分 Angular 组件 / service / interface 样板	人工按现有目录结构重组、重命名并接入真实状态流
算法与可视化实现	明确模式 C 需要真实卷积示例、Grad-CAM；模式 D 需要 Top-K、注意力矩阵和 QKV 教学视图	辅助提供卷积展示、Grad-CAM、attention 读取、QKV 可视化的实现参考	人工按真实推理输出调试数值、修复预处理和资源加载问题
报错排查与资源重构	人工定位模型资源路径、浏览器端运行错误、静态资源同步问题	AI 辅助分析报错原因、给出排查方向和脚本草稿	人工验证资源路径、修正同步方案并确认浏览器端可运行
文案与样式	人工决定页面保留哪些教学文本、哪些开发期提示必须删除	AI 辅助提供中文文案、按钮替换和样式草稿	人工逐项删改并保证与平台已有风格一致

更具体地说：

- 人工主导部分：需求确认、模块范围控制、核心数据结构、接口契约、状态流设计、关键 bug 修复、资源同步方案、与顶栏 / 登录 / AI / 教学入口的集成。
- AI 辅助部分：局部组件样板、接口定义草稿、部分可视化公式参考、报错排查建议、文案和文档组织。
- 最终责任：所有提交到仓库中的模式 C / D 代码、资源路径、运行链验证和答辩叙述，均由成员 C 人工整合、删改、测试和验收。

8. 成员 D：赵红林开发内容

8.1 负责模块概述

成员 D 负责模式 E（反向传播可视化）和模式 F（RNN 循环神经网络）的全部前端开发，并在共享教学词典中新增 12 个术语。

模块	代码路径	核心文件	说明
模式 E: 反向传播可视化	frontend/src/app/modes/mode-e/	engine/mode-e-backprop-engine.ts (654行), services/mode-e-state.service.ts, services/mode-e-assets.service.ts, components/overview/, components/control-panel/, components/detail-panel/, components/floating-charts/, components/shell/	MLP 前向/反向引擎、逐层子步骤动画、神经元权重连线图、决策边界、优化器对比、激活函数切换
模式 F: RNN 循环神经网络	frontend/src/app/modes/mode-f/	engine/mode-f-rnn-engine.ts (180行), services/mode-f-state.service.ts, services/mode-f-assets.service.ts, components/overview/, components/control-panel/, components/detail-panel/, components/shell/	Tanh RNN + BPTT 引擎、时间展开图、三个序列分类数据集
教学术语	frontend/src/app/shared/teaching/teaching-glossary.ts	MODE_E_TEACHING_TERMS (9项), MODE_F_TEACHING_TERMS (8项)	训练与优化分类 + 序列模型分类
AI 助手接入	模式 E Shell	mode-e-explainer-shell.component.ts	模式 E 专属系统提示词、上下文数据构建函数、6 个快捷提问
路由与首页	app.routes.ts, shell/home/	懒加载路由 /mode-e、/mode-f, 首页卡片	紫色/青色主题入口

8.2 公共架构模式

两个模式共用一套 Angular 架构规范:

- **Signal 驱动的状态管理:** 全部 UI 状态使用 `signal()` 和 `computed()`, 共用一个 `@Injectable({ providedIn: 'root' })` 单例状态服务, 不引入 RxJS BehaviorSubject 或 NgRx。
- **纯 TypeScript 计算引擎:** `ModeEBackpropEngine` 和 `ModeFRnnEngine` 不含任何 Angular 依赖, 使用原始 `number[] / number[][]` 运算, 可在浏览器主线程独立运行。
- **懒加载路由:** 页面组件通过 `loadComponent` 动态导入, 首次访问时按需加载整个模式代码块。
- **共享组件复用:** 壳组件统一挂载 `PlatformTopbarComponent` (顶栏)、`LlmFloatingAssistantComponent` (AI 浮窗) 和 `TeachingSearchFabComponent` (教学浮标)。控制面板的按钮通过 `TeachingTermDirective` 接入共享教学文档。
- **I/O 边界:** 所有模式数据均由引擎在内存中计算, 不通过 HTTP 调用后端接口。模式 E 和 F 不与其他模式交换业务数据。

8.3 模式 E: 反向传播可视化

8.3.1 引擎设计

文件: `mode-e-backprop-engine.ts` (654 行)

ModeEBackpropEngine 在浏览器内实现完整的 MLP 训练循环。核心方法及数据流：

方法	输入	输出	说明
forwardPass	layers[], input[]	{ output, cache: ModeEForwardCacheEntry[] }	按拓扑序逐层前向计算。每层记录 preActivation (Z) 和后激活输出 (A) , 供反向传播使用
computeLoss	predictions[], label, lossFunction	{ loss, outputGradient }	支持 crossEntropy、binaryCrossEntropy、mse。输出梯度 dL/dy 直接用于反向传播入口
backwardPass	layers[], forwardCache, outputGradient	ModeELayerGradient[]	逆序遍历层, 先过激活函数导数再过线性变换链式法则, 返回每层 dW、db、梯度范数和统计量
applyGradients	layers[], layerGradients[], config, forwardCache	ModeEParameterSnapshot[]	根据优化器类型累积动量/二阶矩, 原地更新 layer.params , 返回更新前后的参数快照和绝对变化量
trainingStep	以上全部参数 + iteration	ModeEBackpropStep	串联 forward → loss → backward → update, 可选 phaseCallback 用于动画同步
computeDecisionBoundary	layers[], resolution, xRange, yRange	ModeEDecisionBoundary	对二维网格逐点执行 forwardPass, 返回每点的预测类别编号

支持的层类型与反向传播公式：

层类型	前向	反向传播公式
Dense	$Z = X \cdot W^T + b, A = \text{act}(Z)$	$dZ = dA \odot \text{act}'(Z), dW = dZ^T \cdot X, db = \sum dZ, dX = dZ \cdot W$
Activation (ReLU)	$A = \max(0, Z)$	$dZ = dA \odot (Z > 0)$
Activation (Sigmoid)	$A = \sigma(Z)$	$dZ = dA \odot \sigma(Z)(1-\sigma(Z))$
Activation (Tanh)	$A = \tanh(Z)$	$dZ = dA \odot (1-\tanh^2(Z))$
Output (Softmax)	$A = \text{softmax}(Z)$	$dZ = A - y_{\text{onehot}}$ (与 CrossEntropy 合并梯度)

优化器状态管理： optState: ModeEOptimizerState 被所有层共享，按 layerId 键值存储各层的动量 (Momentum) 和一二阶矩估计 (Adam) 。 t 计数器每步递增，用于 Adam 的偏差修正。切换优化器时须调用 reset() 清零状态。

矩阵运算 (均为引擎内部私有函数)： dot (通用矩阵乘法)、 transpose、 hadamard (逐元素乘积)、 addVecToRows (偏置广播)、 softmax (含数值稳定性的 max 减法) 。全部自行实现，不依赖 NumPy、math.js 或 TensorFlow.js。

8.3.2 数据集与预设

文件: mode-e-assets.service.ts

预设 ID	数据集	网络结构	样本数	输出类
xor-mlp	XOR 四团分布	2→12(ReLU)→2(Softmax)	400	2
circle-mlp	同心圆	2→16(Sigmoid)→2(Softmax)	350	2
blobs-mlp	高斯团	2→4(Sigmoid)→3(Softmax)	350	3

数据集在构造时通过静态方法 ModeEBackpropEngine.generateXorData() / generateCircleData() / generateBlobData() 预生成并存入 ModeEDatasetPreset.samples。每个样本为 { input: [x, y], label: number }。

三个预设网络架构是在反复试验后确定的：XOR 用 ReLU 12 单元减少局部最优困局；同心圆用 Sigmoid 16 单元生成弧形边界（ReLU 产生分段直线、无法高效闭合环形）；高斯团用 Sigmoid 4 单元即可线性分割。

8.3.3 状态管理

文件: mode-e-state.service.ts

核心类型 SubStep 将单次训练迭代分解为可手动推进的微步骤：

```
type SubStep =
  | { type: 'idle' }
  | { type: 'forward'; layerPair: number }
  | { type: 'loss' }
  | { type: 'backward'; layerPair: number }
  | { type: 'update'; layerIdx: number }
  | { type: 'done' };
```

buildSubSteps() 根据当前网络层数自动生成完整序列：N-1 个 forward 步骤，1 个 loss 步骤，N-1 个 backward 步骤，K 个 update 步骤（K = 含可训练参数的层数）。用户点击"继续"调用 advanceSubStep() 推进，全部完成后调用 finishAnimation()。

instantStep() 跳过子步骤分拆，直接计算完整训练步——供连续播放模式使用。play() 方法在检测到已达 maxIterations 时自动调用 saveCurrentCurve() 后 pause；再次点击播放会触发 reset() 重新初始化权重后继续。

maybeRecordAvgLoss(itr) 每 25 步遍历全部训练样本，调用引擎的 forwardPass 和 computeLoss 计算平均损失和整体准确率，存入 avgLossHistory（上限 200 条），并触发决策边界重算。

saveCurrentCurve() 将当前 avgLossHistory 快照保存到 savedCurves，标签格式为 \${optimizer}+\${activation} (Acc XX%)，颜色从 8 色调色板中自动选择一个未被使用的。同标签的旧曲线会被替换。

8.3.4 可视化组件

A. 神经元权重连线图（overview/mode-e-overview.component）

SVG 布局常量：LAYER_GAP=240、NEURON_GAP=60、NEURON_R=20。神经元按层排列为垂直列，层间全连接权重边以细线绘制。

子步骤动画的视觉映射：

- 活动层对的连线着色（蓝=forward、橙=backward、绿=update），其余连线保持灰色半透明
- 流动光点沿活动连线做往返 `animateMotion`
- 活动层神经元外圈为对应阶段的彩色环（`hlRingColor`）
- 选中神经元外圈为基于激活符号的辉光环——蓝=正激活、红=负激活（`selGlowColor`）

交互行为：

- 鼠标在 SVG 上移动时，`onSvgMove` 追踪鼠标在 SVG 视口内的像素坐标
- 悬停连线触发 `onEdgeEnter`，设置 `hoveredEdge signal`，`hoverLabel computed` 根据活跃层对判断是否显示浮层
- 浮层为 HTML `<div>` 绝对定位在 SVG 上方，跟随鼠标移动，白色背景黑色等宽字体
- 同层对的其他连线在悬停时透明度降至 0.15（`isEdgeDimmed`）
- 悬停线宽加粗至 2.5px，其余保持 1.2px

B. 浮层图表面板（`floating-charts/mode-e-floating-charts.component`）

`position: fixed` 固定在页面左侧，包含三个子面板：

1. 数据集散点图（点击弹出边界 Modal）：50×50 决策边界半透明色块叠加在数据点下方，数据点坐标通过 `mapX / mapY` 根据实际范围动态映射（含 8% 边距）
2. 损失曲线对比（点击弹出曲线 Modal）：灰色虚线=单步原始损失，实线分别对应当前平滑损失和各已保存曲线。图例旁 x 按钮可删除单条曲线
3. 当前预测：样本坐标、真实标签、预测结果（√/X）

C. 控制面板（`control-panel/mode-e-control-panel.component`）

四列分区布局：网络（预设选择 + 激活函数按钮）、优化器（SGD/Momentum/Adam + 学习率滑块）、训练（步数输入 + 速度按钮）、状态（预测/真实/损失）。训练进行中时（`isRunning` 为 true），除速度按钮外所有配置控件 disabled。

8.3.5 关键类型定义

文件：`mode-e.types.ts`

```
interface ModeEBackpropStep {
  iteration: number; phase: ModeEBackpropPhase; layerIndex: number; totalLayers: number;
  forwardCache?: ModeEForwardCacheEntry[]; loss?: number; predictedClass?: number;
  trueClass?: number; predictions?: number[]; layerGradients: ModeELayerGradient[];
  parameterSnapshots: ModeEParameterSnapshot[]; optimizerState?: ModeEOptimizerState;
}

interface ModeEForwardCacheEntry { layerId: number; layerIndex: number; input: number[][]; output:
number[][]; preActivation?: number[][]; }
interface ModeELayerGradient { layerId: number; weightGradients?: number[][]; biasGradients?:
number[]; inputGradient?: number[][]; gradientNorm: number; gradientStats: { min, max, mean, std
}; }
interface ModeEParameterSnapshot { layerId: number; weightsBefore?: number[][]; weightsAfter?:
number[][]; biasBefore?: number[]; biasAfter?: number[]; weightChange?: number[][]; biasChange?:
number[]; }
interface ModeEDecisionBoundary { resolution: number; xMin: number; xMax: number; yMin: number;
yMax: number; grid: number[][]; }
```

8.4 模式 F: RNN 循环神经网络

8.4.1 引擎设计

文件: mode-f-rnn-engine.ts (约 180 行, 含全部矩阵运算辅助函数)

ModeFRnnEngine 构造函数接收 (inputDim, hiddenDim, outputDim) 初始化三个权重矩阵—— W_{xh} [hiddenDim×inputDim]、 W_{hh} [hiddenDim×hiddenDim]、 W_{hy} [outputDim×hiddenDim]——和两个偏置向量 b_h 、 b_y 。权重初始化为 $[-0.08, 0.08]$ 均匀分布的随机值, 偏置初始化为零。

前向传播: 从 $h_0 = \text{零向量}$ 开始, 对序列的每个时间步 t :

- $h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{t-1} + b_h)$
- $\text{output}_t = \text{softmax}(W_{hy} \cdot h_t + b_y)$
- 返回 ModeFForwardResult { states[], finalPrediction[], predictions[][] }, 包含每个时间步的完整状态快照

BPTT 流程 (trainStep 方法内):

1. 在最后时间步 $T-1$ 计算 softmax 交叉熵梯度 $dy = \text{softmax} - \text{oneHot}(\text{label})$
2. 计算 $dL/dW_{hy} = \sum_t dy_t \otimes h_t$ 和 $dL/db_y = \sum_t dy_t$
3. 将 dy 通过 W_{hy}^T 传播为 dh
4. 从 $T-1$ 迭代到 0, 每步执行:
 - $dh = dh \odot (1 - h_t^2)$ (tanh 导数, 使用 hadamard 逐元素乘积)
 - 累积 $dL/dW_{xh} += dh \otimes x_t$ 、 $dL/dW_{hh} += dh \otimes h_{t-1}$ 、 $dL/db_h += dh$
 - 传播 $dh_{\text{new}} = W_{hh}^T \cdot dh$
5. 累加梯度范数 (用于 UI 展示)

优化器实现: 三个优化器共享同一套 applyUpdate / applyVecUpdate 私有方法, 与模式 E 使用相同的公式 (SGD/Momentum beta=0.9/Adam beta1=0.9 beta2=0.999 eps=1e-8), 但作用于不同形状的权重矩阵。优化器状态作为引擎实例属性而非外部 state 对象管理, 每个权重矩阵有独立的 o (一阶矩) 和 v (二阶矩) 存储。

辅助矩阵函数： dot （通用矩阵乘）、 dotVec （矩阵×向量）、 outer （外积，用于梯度累积）、 add / sub / scale / hprod （基本向量运算）、 softmax （含数值稳定性的 max 减法）。

8.4.2 数据集与预设

文件： mode-f-assets.service.ts

数据集 ID	任务描述	序列长度	输入维度	建议 hiddenDim	样本数
echo	延迟记忆：记住第 0 步的 bit，第 3 步输出分类	4	2	4	200
memory	XOR 记忆：第 0-1 步各给一个 bit，判断是否相同	4	2	4	200
alternation	交替检测：检测前两个 bit 是否发生了交替变化	4	2	6	200

输入采用 one-hot 编码： [bit_value, 0] 表示二进制值。每个数据集由一个私有生成函数构建（ echoDataset / memoryDataset / alternationDataset ），在服务构造时一次性生成并存入 ModeFDatasetPreset.samples 。

8.4.3 状态管理

文件： mode-f-state.service.ts

相比模式 E，状态管理更简——无子步骤动画系统、无决策边界、无激活函数切换、无损失曲线保存对比。训练步是原子的，单步/连续模式直接调用 engine.trainStep() 。

computeAvg() 每 25 步遍历全部样本做纯前向推理（不执行反向传播），计算平均损失和整体准确率。
loadPreset() 创建全新的 ModeFRnnEngine 实例（确保权重完全重置），模式 E 则复用同一个引擎实例并调用 JSON.parse(JSON.stringify(...)) 深拷贝层参数恢复初始权重。

8.4.4 可视化组件

- **Overview：** SVG 时间展开图，每个时间步渲染为一个 Cell 方框（ CELL_W=130, CELL_H=80 ）， Cell 之间用带箭头的连线串联。Cell 内部为隐状态向量的彩色条形图（蓝=正激活、红=负激活、灰=零），下方标注 softmax 输出概率。左侧损失曲线小图显示单步/平滑损失变化。
- **Control Panel：** 预设选择、优化器切换、学习率滑块、步数输入、速度按钮和训练状态显示。训练进行中禁用配置控件。
- **Detail Panel：** 显示权重矩阵形状一览（W_xh/W_hh/W_hy 及偏置维度）、选中时间步的隐状态向量值、梯度范数条形图、每步输出概率分布。

8.5 教学文档贡献

在 teaching-glossary.ts 中新增两项术语数组：

MODE_E_TEACHING_TERMS (9 项, "训练与优化" 分类)： optimizer-sgd （SGD 优化器）、 optimizer-momentum （Momentum 优化器）、 optimizer-adam （Adam 优化器）、 backpropagation （反向传播）、 gradient-descent （梯度下降）、 learning-rate （学习率）、 activation-relu （ReLU）、 activation-sigmoid （Sigmoid）、 activation-tanh （Tanh）。每个术语包含公式、导数、决策边界特征和适用场景说明。

MODE_F_TEACHING_TERMS (8 项, "序列模型" 分类) : rnn-cell (RNN 单元)、bptt (BPTT)、hidden-state (隐状态)、gradient-vanishing (梯度消失)、optimizer-sgd/momentum/adam (优化器术语复用)、sequence-classification (序列分类)。

TeachingTerm 接口的 mode 字段类型已从 'A' | 'C' | 'D' 扩展为 'A' | 'B' | 'C' | 'D' | 'E' | 'F', 合并后的 TEACHING_TERMS 数组通过 findTeachingTerm(id) 全局查找。

模式 E 控制面板的优化器按钮和激活函数按钮通过 [appTeachingTerm] 指令绑定到对应术语 ID, 点击 "?" 浮标激活教学模式后按钮变为浅绿高亮, 点击跳转到 /teaching#termId。

8.6 工程难点

难点	问题背景	解决方案	体现的工程能力
纯 TypeScript 实现完整训练算法	模式 E 需要在不依赖任何数值库的前提下, 完整实现前向传播、损失计算、链式法则反向传播和三种优化器的参数更新, 约 654 行纯手写矩阵运算	手动实现 dot、transpose、hadamard、softmax 等全套矩阵操作。每种激活函数的导数公式 (ReLU/Sigmoid/Tanh) 分别实现并验证与 PyTorch 的结果一致性	算法实现能力、数值计算理解、对深度学习框架底层原理的掌握
子步骤动画状态机设计	需要将单次训练迭代拆分为逐层展示的前向/反向/更新微步骤, 每个微步骤对应特定的 UI 高亮、颜色和流动动画	定义 SubStep 联合类型, 通过 buildSubSteps() 根据实际网络层数动态生成子步骤序列。每个子步骤由 Angular signal 驱动, Overview 组件的 neuronHighlight / edgeOpacity / showFlowDots 等方法根据 SubStep 类型判断当前哪些元素应高亮	状态机设计、Angular Signal 响应式编程、UI 状态与动画的精确同步
决策边界 50×50 网格的性能权衡	每次决策边界更新需对 2500 个网格点各执行一次完整 forwardPass, 3 层网络约 2500×(2×12 + 12×2+12+2) ≈ 170k 次浮点运算	限制更新频率为每 25 步训练后才执行一次。数据点坐标根据实际数据范围 (含 8% margin) 动态映射到 SVG 视口, 防止溢出	性能优化意识、渲染与计算的解耦、边界条件的处理
激活函数选择与决策边界形态的关联	同心圆数据集需要闭合环形边界, 16 个 ReLU 无法高效产生圆弧 (只能拼折线), Sigmoid 16 个单元可以产生弧形山脊拼成闭合环	将同心圆预设改为 Sigmoid 16 隐藏单元。在教学文档中明确解释 ReLU 的分段线性特性和 Sigmoid 的光滑弧形特性, 并设计激活函数切换功能让用户自行对比	对神经网络激活函数特性的深入理解、实验驱动的架构选择
损失曲线对比系统的状态一致性	需要同时维护多条历史曲线 (含优化器+激活标签)、当前运行曲线和原始单步损失曲线, 曲线可单条删除、颜色不重复、同标签自动替换	设计 savedCurves signal + saveCurrentCurve() / deleteSavedCurve(idx) 方法, 8 色调色板自动分配未使用颜色。标签格式 optimizer+activation (Acc XX%) 作为去重键, avgLossHistory 和 lossHistory 分开存储	复杂 UI 状态管理、数据去重与替换逻辑、用户体验设计

难点	问题背景	解决方案	体现的工程能力
RNN BPTT 的时间轴梯度流实现	需要在 4 步序列上正确累积 $W_{xh}/W_{hh}/W_{hy}$ 的梯度, $\tanh$ 导数用隐状态值计算 $(1-h_t^2)$, 不能直接使用 autograd	手动实现 BPTT 循环: 从最后步逆序迭代, 先计算 $\tanh$ 导数 $dh = dh \cdot (1-h^2)$, 再累积外积梯度 $dh \otimes x_t$ 和 $dh \otimes h_{t-1}$, 最后通过 Whh^T 传播到上一步	对 BPTT 算法的完整理解、矩阵运算的维度匹配、数值稳定性处理

8.7 人工与 AI 分工

所有 AI 使用均归属为成员 D 个人贡献的一部分, AI 不作为独立角色。

人工主导:

- 两个引擎的全部数学公式推导和代码实现: MLP 的前向/反向传播链式法则、softmax 交叉熵合并梯度、SGD/Momentum/Adam 的更新规则及偏差修正、RNN 的 BPTT 时间轴梯度累积
- 模式 E 的 SubStep 状态机设计——将训练步拆分为可手动推进的微步骤序列, 子步骤类型的定义和自动生成算法
- 决策边界的渲染策略: 50×50 网格、每 25 步更新、数据范围动态映射 (8% margin 处理溢出)
- 三个预设网络的架构选择: 经过多次实验确定 XOR 用 ReLU 12、同心圆用 Sigmoid 16、高斯团用 Sigmoid 4
- 损失曲线对比系统的去重逻辑 (同标签替换)、颜色分配算法和删除机制
- 控制面板布局设计: 四列分区、训练中禁用逻辑、激活函数切换后的信号强制刷新 (`networkLayers.set([...layers])`)
- 模式 D→E 的代码迁移: 手动解决 git 冲突、修正 19 个恢复文件的路径引用 (从 `features/mode-d-explainer/` 到 `modes/mode-e/`)、修复 import 路径从相对路径到 `@shared/`、`@core/` 别名
- 部署环境验证: 在远端服务器上确认 E/F 路由可访问 (HTTP 200)

AI 辅助:

- 组件样板代码生成: shell、overview、detail-panel、control-panel、floating-charts 的初始文件骨架, 包含 `@Component` 装饰器、standalone imports 和基础模板
- 样式迭代: CSS 颜色值、间距、动画关键帧的调试和微调
- 字符串批量替换: `mode-d` → `mode-e`、`ModeD` → `ModeE`、模式 D → 模式 E 的大规模文件重命名和内容替换
- HTML 模板补全: `@for` / `@if` 控制流代码块、SVG 元素的属性绑定
- 构建错误排查辅助: 识别模板内 `Math.xxx` 不可用、Signal 类型不匹配、`??` 运算符多余等 Angular 编译器警告的具体位置和修复方向
- 教学文档术语的初始文案草稿 (人工修改和补充了公式、导数、决策边界特征等专业内容)